ENTREGA #2

AUTORES:

DANIEL FELIPE SÁNCHEZ PERAFAN
JUAN MANUEL RIVERA
NESTOR DAVID TORRES ROJAS

Presentado a:
JESUS EMILIO PINTO LOPERA
Docente

UNIVERSIDAD DE LA AMAZONIA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
FLORENCIA, CAQUETÁ
2026-2

TABLA DE CONTENIDO

INTRODUCCIÓN 4
METODOLOGÍA 5
Regresión 5
Dataset de regresión y partición de los datos 5
Preprocesamiento según el algoritmo utilizado 6
Regresión lineal bayesiana 6
Máquina de soporte vectorial para regresión 7
Red neuronal artificial: las tres redes 8
Clasificación 9
Dataset de clasificación y partición de los datos 9
Naive Bayes 9
Red Bayesiana 10
Máquina de soporte vectorial 11
Red neuronal artificial: las tres redes 12
Ajuste de los umbrales de clasificación 13
Herramientas utilizadas 13
Métricas de evaluación 13
RESULTADOS 15
Regresión 15
Punto de partida: modelos de la primera entrega 15
Resultados de la regresión bayesiana 15
Resultados de la máquina de soporte vectorial 17
Resultados de las tres redes neuronales 19
Comparación final de los modelos 21
Evaluación sobre el mismo conjunto TEST 21
Comparación común mediante Cross-Validation 23
Datos reales frente a predicciones 23
Clasificación 24
Punto de partida: modelos de la primera entrega 24
Resultados de Naive Bayes 25
Resultados de la Red Bayesiana 28
Resultados de la máquina de soporte vectorial 30

Resultados de las tres redes neuronales 32
Comparación final de modelos 35
CONCLUSIÓN 38

INTRODUCCIÓN

El presente informe corresponde a la segunda entrega del proyecto desarrollado en la asignatura Inteligencia Computacional II y constituye una continuación del trabajo realizado en la primera entrega. Se mantienen los dos problemas de aprendizaje supervisado previamente abordados: uno de regresión, orientado a estimar el tiempo de reacción de jugadores de deportes electrónicos (e-sports) a partir de características biométricas y de comportamiento, y otro de clasificación binaria, cuyo propósito es predecir la posible cancelación de reservas hoteleras a partir de la información disponible sobre cada reserva.

En la primera entrega se implementaron y evaluaron tres algoritmos de aprendizaje automático: Árbol de Decisión, Random Forest y K-Nearest Neighbors (KNN). Para esta segunda fase se amplía el estudio mediante la incorporación de modelos bayesianos, Máquinas de Soporte Vectorial (SVM) y Redes Neuronales Artificiales (RNA). En el problema de regresión se emplean la regresión lineal bayesiana (BayesianRidge), la regresión mediante vectores de soporte (SVR) y redes neuronales de tipo perceptrón multicapa (MLPRegressor). Para el problema de clasificación se implementan Naive Bayes, una Red Bayesiana, Máquinas de Soporte Vectorial y redes neuronales mediante MLPClassifier.

Un componente importante de esta entrega consiste en analizar el efecto de la complejidad de las redes neuronales sobre su desempeño predictivo. Para ello, se construyen tres arquitecturas principales, comenzando con una red de una capa oculta y aumentando progresivamente tanto el número de neuronas como la cantidad de capas. Adicionalmente, se evalúan distintas combinaciones de estos parámetros con el propósito de determinar si una mayor complejidad arquitectónica produce mejoras en la capacidad de generalización o si incrementa el costo computacional sin ofrecer beneficios relevantes.

Con el fin de establecer comparaciones consistentes entre los algoritmos de ambas entregas, se conservan los conjuntos de datos originales y la partición Hold-Out de 70 % para entrenamiento y 30 % para prueba, utilizando las mismas semillas aleatorias. Asimismo, se mantiene la validación cruzada de cinco particiones dentro del conjunto de entrenamiento para evaluar configuraciones y seleccionar hiperparámetros. Debido a las diferencias en los requerimientos computacionales de los

algoritmos, determinadas etapas de selección y entrenamiento se realizan sobre muestras reproducibles del conjunto TRAIN. Estas condiciones se consideran al interpretar los resultados y comparar el desempeño de los modelos.

Finalmente, se recuperan los modelos de la primera entrega y se contrastan sus resultados con los obtenidos mediante los nuevos algoritmos, utilizando métricas apropiadas para cada problema. Además del desempeño predictivo, el análisis considera la estabilidad durante la validación, la complejidad de los modelos, su interpretabilidad y sus requerimientos computacionales. De esta manera, se busca determinar qué enfoques ofrecen un mejor equilibrio entre precisión, capacidad de generalización y eficiencia, manteniendo un procedimiento experimental reproducible y reduciendo el riesgo de fuga de información (data leakage).

METODOLOGÍA

Se siguió el mismo flujo experimental de la primera entrega: preparación de los datos, separación entre entrenamiento y prueba, preprocesamiento dentro de pipelines, selección de hiperparámetros mediante validación cruzada y evaluación final sobre TEST. A este flujo se agregaron los modelos de la segunda entrega y una comparación final que reúne en una misma tabla los modelos de ambas entregas.

Regresión

Dataset de regresión y partición de los datos

Para el desarrollo del problema de regresión se utilizó el mismo conjunto de datos de la primera entrega, contenido en el archivo `data/esports_biometrics.csv`, compuesto por 250.000 registros y 13 columnas. La variable objetivo es `Reaction_Time_ms`, que representa el tiempo de reacción de los jugadores de deportes electrónicos, expresado en milisegundos. Se excluyeron las variables `Player_ID`, debido a que corresponde a un identificador único sin valor predictivo directo, y `Match_Outcome`, porque representa el resultado de la partida y podría introducir información posterior al momento de predicción. De esta manera, se conservaron diez variables predictoras relacionadas con las características biométricas, los hábitos de juego y las condiciones de los jugadores.

Para garantizar la continuidad metodológica con la primera entrega, se conservó el procedimiento de separación Hold-Out, implementado mediante la función `train_test_split` de Scikit-learn. La proporción de datos destinada a prueba se definió mediante el parámetro configurable `HOLDOUT_TEST_SIZE`, que permite establecer distribuciones de entrenamiento y prueba de 60/40, 70/30 u 80/20. Para los experimentos de esta entrega se utilizó el valor `0.30`, correspondiente a una distribución del 70 % para entrenamiento (TRAIN) y el 30 % para prueba (TEST), manteniendo la semilla aleatoria `random_state = 42`. Como resultado, se obtuvieron 175.000 registros de entrenamiento y 75.000 registros de prueba.

La conservación de la misma partición utilizada en la primera entrega permite evaluar los algoritmos anteriores y los nuevos sobre exactamente las mismas observaciones de prueba, favoreciendo una comparación consistente de su desempeño predictivo. El conjunto TEST

permaneció separado durante la selección de hiperparámetros y los procedimientos de validación cruzada, mientras que las transformaciones de preprocesamiento se incorporaron dentro de los pipelines de entrenamiento para reducir el riesgo de fuga de información (*data leakage*). La Figura 1 presenta el fragmento de código correspondiente a la configuración y ejecución de esta partición.

```
# Hold-Out CONFIGURABLE (0.40 -> 60/40, 0.30 -> 70/30, 0.20 -> 80/20)
HOLDOUT_TEST_SIZE = 0.30

target = 'Reaction_Time_ms'
drop_cols = ['Player_ID', 'Match_Outcome']
X = df.drop(columns=drop_cols + [target]).copy()
y = df[target].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=HOLDOUT_TEST_SIZE, random_state=RANDOM_STATE)
```

Figura 1. Partición Hold-Out configurable utilizada en la segunda entrega.

Preprocesamiento según el algoritmo utilizado

El preprocesamiento de los datos se realizó siguiendo los criterios establecidos en la primera entrega, considerando las características de las variables y los requerimientos de cada algoritmo. Para garantizar la reproducibilidad del procedimiento y reducir el riesgo de fuga de información (data leakage), las transformaciones se integraron mediante Pipeline y ColumnTransformer de Scikit-learn. De esta manera, durante la validación cruzada, los parámetros del preprocesamiento se estiman exclusivamente a partir de los registros correspondientes al entrenamiento de cada partición.

En primer lugar, se realizó el tratamiento de los valores faltantes mediante SimpleImputer. Para la variable Screen_Brightness se utilizó la imputación por media, mientras que para HRV_Score se aplicó la mediana. Estas estrategias permiten conservar las observaciones con datos incompletos sin eliminar registros del conjunto de entrenamiento. Por otra parte, las variables categóricas Game_Genre y Blue_Light_Filter se transformaron mediante OneHotEncoder, permitiendo representar sus categorías numéricamente sin establecer relaciones de orden inexistentes.

Se definieron dos procedimientos de preprocesamiento según la naturaleza de los modelos. El primero, implementado mediante la función make_tree_preprocessor(), realiza la imputación y codificación de las variables categóricas sin aplicar escalado a las variables numéricas. Este procedimiento se utiliza en los modelos basados en árboles, como Árbol de Decisión y Random Forest, debido a que sus divisiones se establecen mediante umbrales sobre los atributos y no dependen directamente de la magnitud de sus escalas.

El segundo procedimiento, denominado make_scaled_preprocessor(), incorpora la estandarización de las variables numéricas mediante StandardScaler, además de la imputación y codificación categórica. Este preprocesador se utiliza en KNN, regresión lineal bayesiana, Máquinas de Soporte Vectorial y Redes Neuronales Artificiales. En el caso de KNN, la estandarización evita que los atributos con mayores magnitudes dominen el cálculo de distancias. Para las Máquinas de Soporte Vectorial, contribuye a equilibrar la influencia de los atributos en el cálculo de productos internos o distancias asociadas al kernel. En la

regresión bayesiana, favorece una aplicación más equilibrada de la regularización sobre los coeficientes, mientras que en las redes neuronales facilita el proceso de optimización y puede favorecer la convergencia durante el entrenamiento.

Es importante señalar que la estandarización se aplica a las variables numéricas, mientras que las categóricas se codifican mediante One-Hot Encoding. Asimismo, en el preprocesador escalado se utiliza la opción drop='if_binary', que evita generar dos columnas indicadoras redundantes para las variables categóricas binarias. Todas estas transformaciones se ajustan con los datos de entrenamiento y posteriormente se aplican al conjunto TEST, sin utilizar información de este último para calcular los parámetros del preprocesamiento.

Regresión lineal bayesiana

Para abordar el problema de regresión desde un enfoque probabilístico se implementó el algoritmo de regresión lineal bayesiana mediante BayesianRidge, disponible en Scikit-learn. A diferencia de Naive Bayes, que se utiliza principalmente para problemas de clasificación mediante la estimación de probabilidades de pertenencia a diferentes clases, BayesianRidge permite predecir una variable objetivo continua, como el tiempo de reacción de los jugadores, incorporando un tratamiento probabilístico de los coeficientes del modelo.

La regresión lineal bayesiana establece una relación lineal entre las variables predictoras y la variable objetivo. Sin embargo, a diferencia de una regresión lineal convencional, incorpora distribuciones de probabilidad previas (prior) sobre los coeficientes, generalmente centradas en cero, que representan las suposiciones iniciales del modelo antes de observar los datos. A medida que se incorpora la información del conjunto de entrenamiento, estas distribuciones se actualizan mediante el teorema de Bayes, obteniendo una distribución posterior (posterior) que refleja la información disponible sobre los coeficientes y su incertidumbre.

Una de las principales características de BayesianRidge es que incorpora regularización probabilística, lo que permite controlar la magnitud de los coeficientes y reducir el riesgo de sobreajuste. El algoritmo estima automáticamente la precisión asociada al ruido de las observaciones (α) y la precisión de la distribución de los coeficientes (λ), utilizando un procedimiento iterativo basado en los datos. Para este experimento se utilizó la configuración predeterminada del algoritmo, sin realizar una búsqueda exhaustiva de hiperparámetros mediante GridSearchCV.

El modelo se implementó mediante un Pipeline que integra el preprocesamiento escalado y el estimador BayesianRidge. Su desempeño se evaluó utilizando validación cruzada de cinco particiones (5-fold Cross-Validation) sobre una muestra reproducible de 8.000 registros del conjunto TRAIN, empleando las métricas RMSE, MAE y R^2. Posteriormente, se entrenó el modelo final con los 175.000 registros del conjunto de entrenamiento y se evaluó sobre los 75.000 registros reservados para TEST.

Adicionalmente, se aprovechó la capacidad del algoritmo para estimar la incertidumbre asociada a sus predicciones. Mediante el parámetro return_std=True del método predict(), se obtuvo tanto el valor esperado del tiempo de reacción como su desviación estándar predictiva. A partir de esta información se construyeron intervalos predictivos aproximados del 95 %, calculados mediante la predicción ± 1,96 veces la desviación estándar, bajo los supuestos probabilísticos del modelo. Esto permite complementar la evaluación del error con una estimación de la incertidumbre de cada predicción.

La Figura 2 presenta la implementación del modelo de regresión lineal bayesiana y el procedimiento empleado para obtener sus predicciones junto con la incertidumbre estimada.

```python
bayes_model = Pipeline([
    ('preprocessor', make_scaled_preprocessor()),
    ('model', BayesianRidge()),
])
bayes_model.fit(X_train, y_train)

# Prediccion + incertidumbre (desviacion estandar)
Xte = bayes_model.named_steps['preprocessor'].transform(X_test)
mean_pred, std_pred = bayes_model.named_steps['model'].predict(
    Xte, return_std=True)
```

Figura 2. Regresión bayesiana: entrenamiento y predicción con incertidumbre.

Máquina de soporte vectorial para regresión

Para abordar la estimación del tiempo de reacción mediante Máquinas de Soporte Vectorial se implementó el algoritmo Support Vector Regression (SVR), disponible en Scikit-learn. Este método busca construir una función predictiva que mantenga los errores dentro de una banda de tolerancia definida por el parámetro ε (epsilon), procurando simultáneamente controlar la complejidad del modelo. A diferencia de otros algoritmos de regresión que penalizan directamente todas las desviaciones entre valores reales y predichos, SVR utiliza una función de pérdida ε-insensible, en la cual los errores cuyo valor absoluto no supera ε no generan penalización.

El funcionamiento del algoritmo depende de tres elementos principales. El parámetro epsilon determina el margen de tolerancia alrededor de la función predictiva; el parámetro C controla la penalización de las desviaciones que superan dicho margen y establece un equilibrio entre el ajuste a los datos y la regularización; mientras que kernel determina la forma de representar las relaciones entre las variables. Para este experimento se evaluaron dos tipos de kernel: lineal (linear), apropiado para representar relaciones lineales, y de función de base radial (RBF), que permite modelar relaciones no lineales. En la formulación SVR, los vectores de soporte corresponden a observaciones que contribuyen a definir la función de predicción, generalmente ubicadas sobre los límites de la banda de tolerancia o fuera de ella.

La selección de hiperparámetros se realizó mediante GridSearchCV, evaluando 18 configuraciones resultantes de combinar los kernels lineal y RBF, los valores C = {1, 10, 100} y epsilon = {0.5, 2, 5}. Cada configuración se evaluó mediante validación cruzada de cinco particiones (5-fold Cross-Validation) sobre una muestra reproducible de 8.000 registros del

conjunto TRAIN, utilizando RMSE como criterio principal de selección. El preprocesamiento se integró mediante un Pipeline que incorpora imputación, codificación de variables categóricas y estandarización de variables numéricas.

Debido a que las implementaciones de SVR basadas en kernels pueden presentar un costo computacional elevado cuando aumenta la cantidad de observaciones, se establecieron estrategias de entrenamiento diferentes según el kernel seleccionado. Para el kernel RBF se contempló utilizar una muestra de 30.000 registros de TRAIN en el ajuste final. En cambio, para el kernel lineal se optó por utilizar LinearSVR, una implementación optimizada para problemas lineales que permite trabajar de manera más eficiente con conjuntos de datos de mayor tamaño. Es importante señalar que LinearSVR y SVR(kernel='linear') no son implementaciones matemáticamente idénticas, por lo que la selección de hiperparámetros realizada con SVR constituye una referencia para la configuración del modelo lineal final, pero no garantiza que sea la configuración óptima de LinearSVR.

En el experimento, la validación cruzada seleccionó el kernel lineal con C = 1 y epsilon = 5. En consecuencia, el modelo final se implementó mediante LinearSVR y se entrenó utilizando los 175.000 registros del conjunto TRAIN. Posteriormente, se evaluó sobre los 75.000 registros de TEST, manteniendo este conjunto separado durante las etapas de selección y entrenamiento.

La Figura 3 presenta el procedimiento de búsqueda de hiperparámetros utilizado para evaluar las diferentes configuraciones de SVR mediante validación cruzada.

```
svr_grid = GridSearchCV(
    Pipeline([('preprocessor', make_scaled_preprocessor()),
              ('model', SVR())]),
    param_grid={'model__kernel': ['linear', 'rbf'],
                'model__C': [1, 10, 100],
                'model__epsilon': [0.5, 2, 5]},
    scoring=scoring, refit='RMSE', cv=cv5, n_jobs=-1)
svr_grid.fit(X_svr_cv, y_svr_cv)
```

Figura 3. Búsqueda de kernel, C y ε de la SVR mediante validación cruzada.

Red neuronal artificial: las tres redes

Para el problema de regresión se implementaron Redes Neuronales Artificiales mediante el algoritmo `MLPRegressor` (Multi-Layer Perceptron), disponible en Scikit-learn. Este modelo está compuesto por neuronas artificiales organizadas en capas, las cuales procesan la información mediante conexiones ponderadas por pesos y términos de sesgo. Cada neurona recibe las salidas de la capa anterior y aplica una función de activación que permite representar relaciones no lineales entre las variables predictoras y el tiempo de reacción. Durante el entrenamiento, los pesos se ajustan mediante retropropagación del error y procedimientos de optimización, con el objetivo de minimizar la función de pérdida.

Para analizar el efecto de la complejidad arquitectónica sobre el desempeño predictivo, se construyeron tres redes neuronales con diferentes cantidades de capas ocultas y neuronas. La primera, denominada red suave, contiene una capa oculta de 16 neuronas; la segunda, red intermedia, incorpora dos capas ocultas de 64 neuronas cada una; y la tercera, red grande, está formada por tres capas ocultas de 128 neuronas cada una. En estas configuraciones se incrementan simultáneamente la profundidad y el número de neuronas, lo que permite estudiar el comportamiento de modelos progresivamente más complejos. Las arquitecturas seleccionadas se presentan en la Tabla 1.

Red	**Capas ocultas**	**Neuronas por capa**	**Arquitectura**
Suave	1	16	(16)
Intermedia	2	64	(64, 64)
Grande	3	128	(128, 128, 128)

Tabla 1. Arquitecturas de las tres redes neuronales.

Las tres redes neuronales se entrenaron bajo las mismas condiciones experimentales, utilizando la función de activación ReLU (Rectified Linear Unit) y el optimizador Adam, encargado de actualizar los pesos durante el aprendizaje. Se estableció un máximo de 300 iteraciones mediante `max_iter=300` y se incorporó el mecanismo de detención temprana (*early stopping*) para reducir el riesgo de sobreajuste. Este procedimiento reserva internamente el 10 % de los registros disponibles para el entrenamiento de cada red como conjunto de validación y permite detener el

proceso cuando no se observa una mejora suficiente durante diez iteraciones consecutivas (`n_iter_no_change=10`).

El preprocesamiento de las variables predictoras se realizó mediante `make_scaled_preprocessor()`, que integra la imputación de valores faltantes, la codificación de variables categóricas y la estandarización de atributos numéricos. Adicionalmente, se utilizó `TransformedTargetRegressor` junto con `StandardScaler` para estandarizar la variable objetivo `Reaction_Time_ms` durante el entrenamiento. Esta transformación facilita la optimización del modelo al trabajar con valores de salida estandarizados. Posteriormente, las predicciones se transforman nuevamente a la escala original, permitiendo expresar los resultados en milisegundos.

Para evaluar el desempeño de las arquitecturas se aplicó validación cruzada de cinco particiones (*5-fold Cross-Validation*) sobre una muestra reproducible de 20.000 registros del conjunto TRAIN. Se utilizaron las métricas RMSE, MAE y R^2 para analizar el error de predicción y la capacidad explicativa de cada configuración. Posteriormente, cada red se entrenó con los 175.000 registros del conjunto TRAIN y se evaluó sobre los 75.000 registros de TEST. Esta estrategia permitió comparar las arquitecturas bajo condiciones comunes de evaluación y analizar su comportamiento sobre observaciones no utilizadas durante el entrenamiento.

La comparación entre arquitecturas se realizó principalmente mediante el RMSE medio obtenido en validación cruzada, considerando también la desviación estándar entre folds. Como criterio descriptivo, las diferencias de RMSE inferiores a la desviación estándar media observada se clasificaron como resultados similares o sin cambios relevantes. No obstante, este procedimiento constituye una regla práctica de comparación y no una prueba formal de equivalencia o significancia estadística. También se consideraron el número de conexiones, las épocas de entrenamiento y el costo computacional para evaluar el equilibrio entre complejidad y desempeño.

La Figura 4 presenta la implementación de las tres arquitecturas neuronales y la configuración utilizada durante su entrenamiento y evaluación.

```python
THREE_NETS = {
    'Red neuronal suave (16)': (16,),
    'Red neuronal intermedia (64, 64)': (64, 64),
    'Red neuronal grande (128, 128, 128)': (128, 128, 128),
}

def make_mlp(hidden):
    return Pipeline([
        ('preprocessor', make_scaled_preprocessor()),
        ('model', TransformedTargetRegressor(
            regressor=MLPRegressor(hidden_layer_sizes=hidden,
                                   activation='relu', solver='adam',
                                   early_stopping=True, max_iter=300,
                                   random_state=RANDOM_STATE),
            transformer=StandardScaler()))])
```

Figura 4. Definición de las tres redes neuronales de regresión.

Como análisis complementario, se evaluaron nueve configuraciones adicionales de arquitectura, resultantes de combinar una, dos y tres capas ocultas con 16, 64 y 128 neuronas por capa. Estas combinaciones incluyen las tres arquitecturas principales y permiten analizar por separado el efecto de aumentar la profundidad de la red y el de incrementar el número de neuronas, manteniendo constante el otro factor. Todas las configuraciones se evaluaron mediante el mismo procedimiento de validación cruzada, con el propósito de identificar si las variaciones de complejidad producen cambios relevantes en el desempeño predictivo. Los resultados de estos experimentos se presentan posteriormente en la sección de resultados.

Clasificación

Dataset de clasificación y partición de los datos

Para el desarrollo del problema de clasificación se utilizó el conjunto de datos de reservas hoteleras preparado durante la primera entrega, almacenado en el archivo data/hotel_bookings_features.csv. Este conjunto contiene 119.390 registros y 29 columnas, de las cuales 28 corresponden a variables predictoras y una a la variable objetivo is_canceled. Esta última identifica si una reserva fue cancelada (clase 1) o no cancelada (clase 0), por lo que el problema se aborda mediante técnicas de clasificación binaria.

El conjunto de datos incluye características relacionadas con las condiciones de la reserva, la anticipación con la que se realiza, el tipo de depósito, el segmento de mercado, el historial de cancelaciones, las solicitudes especiales y otras variables asociadas al

comportamiento de los clientes. Se conservó la depuración realizada en la primera entrega, en la cual se excluyeron atributos como reservation_status, reservation_status_date y assigned_room_type, debido a que podrían proporcionar información posterior al momento en que se pretende realizar la predicción, generando riesgo de fuga de información (*data leakage*).

La distribución de la variable objetivo presenta un desbalance moderado entre las clases: aproximadamente el 62,96 % de los registros corresponde a reservas no canceladas y el 37,04 % a reservas canceladas. Esta diferencia debe considerarse durante la evaluación, debido a que un modelo puede obtener una exactitud global relativamente elevada favoreciendo la clase mayoritaria, sin necesariamente identificar adecuadamente las cancelaciones. Por esta razón, además de Accuracy, se utilizan métricas como Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y Average Precision, que permiten analizar diferentes aspectos del desempeño predictivo.

Para mantener la continuidad metodológica con la primera entrega, se implementó una separación Hold-Out mediante train_test_split, utilizando el parámetro configurable HOLDOUT_TEST_SIZE = 0.30 y la semilla aleatoria random_state = 42. Adicionalmente, se empleó la opción stratify=y, con el propósito de conservar aproximadamente la misma proporción de reservas canceladas y no canceladas en los conjuntos de entrenamiento y prueba. Como resultado, se obtuvieron 83.573 registros para TRAIN y 35.817 para TEST.

La conservación de los mismos datos y de la misma partición permite comparar los algoritmos implementados en ambas entregas sobre un conjunto común de observaciones. Los procedimientos de selección de hiperparámetros y ajuste de umbrales se desarrollan dentro de TRAIN, mediante validación cruzada estratificada de cinco particiones (StratifiedKFold), mientras que TEST se mantiene separado para la evaluación final de los modelos. Esta estrategia busca reducir el riesgo de sobreajuste y evitar que la información del conjunto de prueba influya directamente en las decisiones de entrenamiento.

Naive Bayes

Para abordar el problema de clasificación desde un enfoque probabilístico se implementó el algoritmo Naive Bayes, fundamentado en el teorema de Bayes. Este método permite estimar la

probabilidad de que una reserva hotelera pertenezca a una determinada clase, cancelada o no cancelada, a partir de sus características. Su funcionamiento combina la probabilidad previa de cada clase (*prior*) con la información proporcionada por las variables predictoras, permitiendo obtener una estimación probabilística de la categoría correspondiente a cada observación.

La principal característica de Naive Bayes es que asume independencia condicional entre los atributos, es decir, considera que, una vez conocida la clase, el valor de una variable predictora no depende de los valores de las demás. Bajo este supuesto, la probabilidad conjunta de los atributos condicionada a una clase puede expresarse como el producto de sus probabilidades individuales. Aunque esta condición no necesariamente se cumple en conjuntos de datos reales, permite simplificar considerablemente el cálculo probabilístico y construir modelos computacionalmente eficientes.

$$P(cancela \mid x_1, \ldots, x_n) \propto P(cancela) \cdot P(x_1 \mid cancela) \cdot P(x_2 \mid cancela) \cdot \ldots \cdot P(x_n \mid cancela)$$

En este problema, la probabilidad previa de cancelación corresponde aproximadamente al 37,04 %, mientras que la de no cancelación es del 62,96 %. Estas proporciones representan la distribución inicial de las clases antes de considerar las características particulares de cada reserva. Durante el entrenamiento, el algoritmo estima las probabilidades o distribuciones condicionales de los atributos para cada clase, utilizando posteriormente esta información para calcular la probabilidad de cancelación de nuevas observaciones.

Antes de evaluar los modelos, se realizó un análisis exploratorio de la dependencia entre atributos mediante la V de Cramér, medida estadística que cuantifica la intensidad de asociación entre variables categóricas. Para este procedimiento se utilizaron once atributos seleccionados y discretizados, calculando las asociaciones por separado dentro de cada clase y obteniendo un promedio ponderado según la cantidad de observaciones. Este análisis permitió identificar relaciones entre determinadas características incluso después de considerar la clase de cancelación. Por tanto, se utilizó como evidencia descriptiva para examinar el supuesto de independencia condicional de Naive Bayes, sin interpretarlo como una prueba exhaustiva de independencia de todas las variables del conjunto de datos.

Debido a que el dataset contiene características numéricas y categóricas, se evaluaron dos variantes del algoritmo. La primera fue GaussianNB, que modela cada atributo mediante una distribución normal condicionada a la clase. Para su implementación se utilizó un pipeline con imputación de valores faltantes, estandarización de variables numéricas y codificación de variables categóricas mediante One-Hot Encoding. Se evaluaron cuatro configuraciones del parámetro var_smoothing: 10^{-9}, 10^{-6}, 10^{-3} y 10^{-1}, con el propósito de analizar el efecto de la estabilización numérica de las varianzas.

La segunda variante fue CategoricalNB, diseñada para trabajar con atributos representados mediante categorías discretas. En este caso, las variables numéricas se transformaron en intervalos utilizando KBinsDiscretizer con una estrategia basada en cuantiles, mientras que las variables categóricas se codificaron mediante identificadores enteros. Adicionalmente, se agruparon las categorías menos frecuentes de la variable country y se incorporó un tratamiento para categorías no observadas durante el entrenamiento. Se evaluaron configuraciones con cinco y diez intervalos de discretización (n_bins) y valores de suavizado alpha de 0,5 y 1,0.

Ambas variantes se evaluaron mediante validación cruzada estratificada de cinco particiones (StratifiedKFold) sobre los 83.573 registros del conjunto TRAIN. Las transformaciones de preprocesamiento se incorporaron dentro de los respectivos pipelines, de manera que sus parámetros se ajustaran exclusivamente con los datos de entrenamiento de cada fold, evitando la utilización de información procedente de las particiones de validación.

Para seleccionar la variante más adecuada se utilizó ROC-AUC como criterio principal, debido a que permite evaluar la capacidad del modelo para discriminar entre reservas canceladas y no canceladas sin depender de un único umbral de clasificación. Posteriormente, el umbral de decisión se ajustó mediante predicciones fuera de muestra (*out-of-fold*) obtenidas dentro de TRAIN, buscando maximizar el F1-score. Finalmente, el modelo seleccionado se entrenó con el conjunto TRAIN completo y se evaluó sobre TEST, manteniendo la independencia de este último durante la selección y el ajuste.

La Figura 5 presenta el pipeline utilizado para el preprocesamiento y entrenamiento de la

variante categórica de Naive Bayes.

```
def make_categorical_nb(n_bins=5, alpha=1.0):
    pre = ColumnTransformer([
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('bins', KBinsDiscretizer(n_bins=n_bins, encode='ordinal',
                                      strategy='quantile'))]), numeric_cols),
        ('country', Pipeline([('bucket', TopCategoryBucketer(top_n=15)),
                              ('ord', SafeOrdinalEncoder())]), ['country']),
        ('cat', SafeOrdinalEncoder(), other_cat),
    ])
    return Pipeline([('preprocessor', pre),
                     ('model', CategoricalNB(alpha=alpha))])
```

Figura 5. Pipeline de Naive Bayes categórico.

Red Bayesiana

Como complemento de Naive Bayes se construyó una Red Bayesiana con la biblioteca pgmpy. Mientras Naive Bayes impone una estructura fija —la clase influye en todos los atributos y ningún atributo se relaciona con otro—, una Red Bayesiana es un grafo dirigido en el que cada flecha A → B indica que B depende directamente de A. La biblioteca aprende ese grafo a partir de los datos y permite identificar qué atributos son dependientes de cuáles.

La estructura se obtuvo mediante una búsqueda Hill Climbing, que agrega, elimina o invierte flechas mientras mejore la puntuación BIC; esta puntuación premia el ajuste a los datos y penaliza los grafos demasiado complejos. Luego se estimó para cada nodo una tabla de probabilidades condicionales con un prior BDeu, como se muestra en la figura 6. Las redes bayesianas trabajan con variables discretas, por lo que se utilizaron once atributos discretizados: deposit_type, lead_time, country, market_segment, total_of_special_requests, previous_cancellations, required_car_parking_spaces, customer_type, booking_changes, hotel_type y adr. Para que la comparación con Naive Bayes fuera justa, también se entrenó un Naive Bayes con exactamente esos mismos once atributos.

```python
from pgmpy.estimators import HillClimbSearch, BayesianEstimator
from pgmpy.models import DiscreteBayesianNetwork

# 1) Estructura: busqueda Hill Climbing con puntuacion BIC
dag = HillClimbSearch(bn_train).estimate(scoring_method='bic-d',
                                         max_indegree=3)
# 2) Parametros: tablas de probabilidad condicional (prior BDeu)
bn_model = DiscreteBayesianNetwork(dag.edges())
cpds = BayesianEstimator(bn_model, bn_train).get_parameters(
    prior_type='BDeu', equivalent_sample_size=10)
bn_model.add_cpds(*cpds)
```

Figura 6. Aprendizaje de la estructura y de los parámetros de la Red Bayesiana.

Máquina de soporte vectorial

Para abordar el problema de clasificación de reservas hoteleras se implementaron Máquinas de Soporte Vectorial (*Support Vector Machines*, SVM), utilizando las clases LinearSVC y SVC de la biblioteca Scikit-learn. Este algoritmo busca establecer una frontera de decisión que permita diferenciar las reservas canceladas de las no canceladas, procurando maximizar el margen de separación entre ambas clases y controlar los errores de clasificación. Las observaciones que intervienen directamente en la determinación de esta frontera se denominan vectores de soporte.

El funcionamiento de las SVM está condicionado por la selección del kernel y los parámetros de regularización. Para este experimento se evaluaron dos enfoques: uno lineal, implementado mediante LinearSVC, que establece una frontera de decisión mediante un hiperplano; y otro no lineal, implementado mediante SVC con kernel de función de base radial (RBF), que permite representar fronteras de decisión más complejas mediante relaciones de similitud entre observaciones. En este último se utilizó gamma='scale', configuración que determina automáticamente el parámetro del kernel a partir de las características de los datos transformados.

El parámetro C controla la intensidad de la regularización y establece un equilibrio entre la amplitud del margen y la penalización de los errores de clasificación. Los valores elevados de C penalizan con mayor intensidad los errores, mientras que los valores pequeños permiten una mayor tolerancia a clasificaciones incorrectas a cambio de una regularización más fuerte. Adicionalmente, se evaluó class_weight='balanced', que ajusta la penalización asignada a cada

clase de acuerdo con su frecuencia en el conjunto de entrenamiento. Esta estrategia permite considerar el desbalance entre reservas canceladas y no canceladas sin modificar directamente la cantidad de registros de cada categoría.

La selección de hiperparámetros se realizó mediante GridSearchCV, utilizando validación cruzada estratificada de cinco particiones (*5-fold Stratified Cross-Validation*) sobre una muestra reproducible de 8.000 registros del conjunto TRAIN. Para el modelo lineal se evaluaron los valores C = {0.01, 0.1, 1}, mientras que para el kernel RBF se consideraron C = {0.1, 1, 10}. En ambos casos se evaluaron las configuraciones class_weight=None y class_weight='balanced', obteniendo un total de doce combinaciones. La selección se realizó utilizando el F1-score como métrica principal, junto con Precision, Recall y ROC-AUC como indicadores complementarios.

El preprocesamiento de las variables se integró mediante un Pipeline, utilizando imputación de valores faltantes, codificación de variables categóricas mediante One-Hot Encoding y estandarización de variables numéricas con StandardScaler. Estas transformaciones se ajustaron dentro de cada partición de entrenamiento de la validación cruzada, con el propósito de evitar que los datos de validación influyeran en los parámetros del preprocesamiento.

Debido al elevado costo computacional de las SVM con kernel RBF cuando aumenta la cantidad de observaciones, se establecieron estrategias diferenciadas para el entrenamiento final. Si la mejor configuración correspondía al modelo lineal, se utilizaría LinearSVC sobre los 83.573 registros de TRAIN. Por el contrario, si se seleccionaba el kernel RBF, el entrenamiento se realizaría sobre una muestra estratificada y reproducible de 30.000 registros de TRAIN. Esta decisión permite reducir los requerimientos computacionales, aunque implica que el modelo no aprovecha todas las observaciones disponibles durante su ajuste final.

Para determinar el umbral de clasificación se utilizó cross_val_predict con el método decision_function, obteniendo puntuaciones fuera de muestra (*out-of-fold*) mediante validación cruzada sobre los 8.000 registros empleados en la selección de hiperparámetros. A diferencia de una probabilidad estimada, estas puntuaciones representan la posición relativa de las observaciones respecto a la frontera de decisión del clasificador. Se evaluaron diferentes umbrales obtenidos a partir de cuantiles de dichas puntuaciones y se seleccionó aquel que maximizó el F1-score dentro del conjunto de entrenamiento.

Finalmente, el modelo seleccionado se entrenó con la muestra de TRAIN correspondiente a su configuración y se evaluó sobre los 35.817 registros del conjunto TEST. Para esta evaluación se utilizaron las métricas Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y Average Precision. Las métricas dependientes del umbral se calcularon utilizando el valor seleccionado mediante validación cruzada, mientras que ROC-AUC y Average Precision se calcularon directamente a partir de las puntuaciones de decisión.

La Figura 7 presenta el procedimiento implementado para la búsqueda y selección de hiperparámetros de las Máquinas de Soporte Vectorial mediante validación cruzada.

```
svm_grid = GridSearchCV(svm_pipe, param_grid=[
    {'model': [LinearSVC(dual='auto', max_iter=10000)],
     'model__C': [0.01, 0.1, 1],
     'model__class_weight': [None, 'balanced']},
    {'model': [SVC(kernel='rbf', gamma='scale')],
     'model__C': [0.1, 1, 10],
     'model__class_weight': [None, 'balanced']},
], scoring=SCORING, refit='f1', cv=cv5, n_jobs=-1)
svm_grid.fit(X_svm_cv, y_svm_cv)
```

Figura 7. Búsqueda de kernel, C y class_weight de la SVM.

Red neuronal artificial: las tres redes

Para abordar el problema de clasificación binaria se implementaron Redes Neuronales Artificiales mediante el algoritmo MLPClassifier (*Multi-Layer Perceptron*), disponible en la biblioteca Scikit-learn. Este modelo está compuesto por neuronas artificiales organizadas en capas, que procesan las variables predictoras mediante conexiones ponderadas por pesos y términos de sesgo. Cada neurona combina la información recibida de la capa anterior y aplica una función de activación, permitiendo aprender relaciones no lineales entre las características de las reservas hoteleras. En la clasificación binaria, la capa de salida utiliza una función logística para estimar la probabilidad de que una reserva pertenezca a la clase positiva, correspondiente a una reserva cancelada.

Con el propósito de analizar la influencia de la complejidad arquitectónica sobre el desempeño predictivo, se utilizaron las mismas tres configuraciones principales definidas en el problema de regresión, presentadas en la Tabla 1. La primera, denominada red suave, contiene una capa oculta de 16 neuronas; la segunda, red intermedia, incorpora dos capas ocultas de 64

neuronas cada una; y la tercera, red grande, está formada por tres capas ocultas de 128 neuronas cada una. Esta organización permite comparar arquitecturas progresivamente más complejas y analizar si el aumento simultáneo de capas ocultas y neuronas produce mejoras en la capacidad de clasificación.

Las tres redes se implementaron bajo las mismas condiciones experimentales, utilizando la función de activación ReLU (*Rectified Linear Unit*) en las capas ocultas y el optimizador Adam para actualizar los pesos mediante retropropagación del error. Se estableció un máximo de 300 iteraciones (max_iter=300) y se incorporó el mecanismo de detención temprana (early_stopping=True), reservando internamente el 10 % de los datos de entrenamiento para validación (validation_fraction=0.1). El procedimiento permite detener el aprendizaje cuando la exactitud (*Accuracy*) de validación interna no presenta una mejora suficiente durante diez iteraciones consecutivas (n_iter_no_change=10), contribuyendo a controlar el sobreajuste y el costo computacional.

El preprocesamiento se integró mediante un Pipeline, utilizando la función build_preprocessors() con la opción scale_numeric=True. Este procedimiento incorpora la imputación de valores faltantes, el tratamiento de determinadas variables y valores extremos, la codificación de variables categóricas mediante One-Hot Encoding y la estandarización de las variables numéricas con StandardScaler. Las transformaciones se ajustaron exclusivamente con los datos de entrenamiento correspondientes a cada partición, evitando que los registros de validación o prueba intervinieran en la estimación de sus parámetros.

Para evaluar el desempeño de las arquitecturas se utilizó validación cruzada estratificada de cinco particiones (StratifiedKFold) sobre una muestra reproducible de 20.000 registros del conjunto TRAIN. Durante este procedimiento se calcularon F1-score y ROC-AUC, considerando sus valores medios y la variabilidad entre los folds. En particular, el F1-score de esta comparación se obtuvo utilizando el umbral predeterminado de clasificación de 0,5, permitiendo analizar las arquitecturas bajo un mismo criterio inicial de decisión.

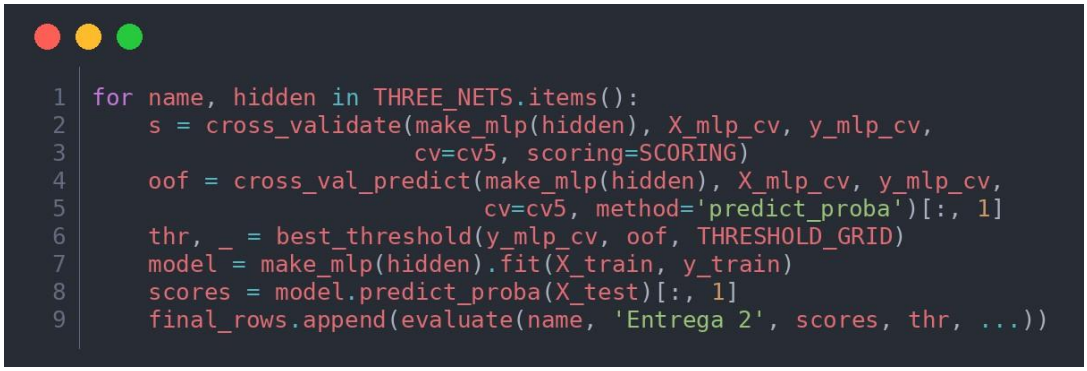
Posteriormente, para cada arquitectura se generaron probabilidades de clasificación fuera de muestra (*out-of-fold*) mediante cross_val_predict, utilizando las mismas cinco particiones de

validación cruzada. A partir de estas probabilidades se evaluaron umbrales entre 0,10 y 0,90, con incrementos de 0,01, y se seleccionó aquel que maximizó el F1-score. Este procedimiento permitió ajustar el criterio de clasificación de cada red utilizando exclusivamente información del conjunto TRAIN, sin emplear los registros de TEST para seleccionar los umbrales.

Una vez establecidos los procedimientos de evaluación y los umbrales de decisión, cada red neuronal se entrenó nuevamente utilizando los 83.573 registros del conjunto TRAIN completo y se evaluó sobre los 35.817 registros de TEST. Para esta evaluación se utilizaron Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y Average Precision. Las métricas dependientes del umbral se calcularon con el punto de corte seleccionado mediante validación cruzada, mientras que ROC-AUC y Average Precision se obtuvieron directamente a partir de las probabilidades estimadas.

Como análisis complementario, se evaluaron nueve combinaciones de arquitectura, incluidas las tres redes principales, resultantes de combinar una, dos y tres capas ocultas con 16, 64 y 128 neuronas por capa. Este procedimiento permitió analizar por separado el efecto de incrementar la profundidad de la red y el número de neuronas, manteniendo constante el otro factor. Las configuraciones se compararon mediante el F1-score medio de validación cruzada y su desviación estándar, considerando también la cantidad de conexiones y el costo computacional. Las diferencias inferiores a la desviación estándar media entre folds se interpretaron descriptivamente como resultados similares, sin asumir que esto constituye una prueba formal de equivalencia estadística.

La Figura 8 presenta el procedimiento utilizado para evaluar las tres arquitecturas neuronales, seleccionar sus umbrales de clasificación y obtener las métricas correspondientes sobre el conjunto TEST.



```
for name, hidden in THREE_NETS.items():
    s = cross_validate(make_mlp(hidden), X_mlp_cv, y_mlp_cv,
                       cv=cv5, scoring=SCORING)
    oof = cross_val_predict(make_mlp(hidden), X_mlp_cv, y_mlp_cv,
                            cv=cv5, method='predict_proba')[:, 1]
    thr, _ = best_threshold(y_mlp_cv, oof, THRESHOLD_GRID)
    model = make_mlp(hidden).fit(X_train, y_train)
    scores = model.predict_proba(X_test)[:, 1]
    final_rows.append(evaluate(name, 'Entrega 2', scores, thr, ...))
```

Figura 8. Evaluación de las tres redes neuronales de clasificación.

Ajuste de los umbrales de clasificación

Para la evaluación de los modelos de clasificación se mantuvo el procedimiento de ajuste de umbrales establecido en la primera entrega, utilizando el F1-score como criterio principal. El umbral de decisión determina a partir de qué valor de probabilidad o puntuación una observación se clasifica como reserva cancelada (clase 1). Aunque en los clasificadores probabilísticos suele utilizarse un valor predeterminado de 0,5, este no necesariamente proporciona el mejor equilibrio entre Precision y Recall, especialmente cuando existe una distribución desigual entre las clases.

La selección de los umbrales se realizó utilizando exclusivamente datos del conjunto TRAIN, mediante predicciones fuera de muestra (*out-of-fold*) obtenidas con validación cruzada estratificada de cinco particiones. Este procedimiento permite generar una predicción para cada observación a partir de un modelo que no fue entrenado con dicho registro. Posteriormente, mediante la función best_threshold(), se evaluaron diferentes valores de corte y se seleccionó aquel que produjo el mayor F1-score.

Para Naive Bayes, la Red Bayesiana y las redes neuronales se utilizaron como candidatos los umbrales comprendidos entre 0,10 y 0,90, con incrementos de 0,01. En el caso de las Máquinas de Soporte Vectorial, el ajuste se realizó sobre las puntuaciones generadas mediante decision_function, empleando 181 valores candidatos obtenidos a partir de cuantiles entre el 5 % y el 95 % de las puntuaciones fuera de muestra. Estas puntuaciones no representan probabilidades calibradas, por lo que sus umbrales deben interpretarse como puntos de corte de la función de decisión y no como probabilidades de cancelación.

Una vez seleccionado el umbral de cada modelo, este se mantuvo fijo durante la evaluación final sobre TEST. Las métricas Accuracy, Balanced Accuracy, Precision, Recall y F1-score se calcularon utilizando las etiquetas obtenidas mediante el umbral correspondiente. Por su parte, ROC-AUC y Average Precision se calcularon directamente a partir de las probabilidades estimadas o las puntuaciones de decisión, sin aplicar dichos puntos de corte, permitiendo evaluar la capacidad discriminativa de los modelos.

Los clasificadores recuperados de la primera entrega conservaron sus umbrales previamente seleccionados, mientras que los nuevos modelos utilizaron los valores determinados mediante los procedimientos descritos. Es importante señalar que, debido a los requerimientos computacionales de cada algoritmo, las predicciones utilizadas para ajustar los umbrales no siempre se obtuvieron a partir del mismo número de registros de TRAIN. Esta diferencia debe considerarse al comparar el desempeño de las configuraciones evaluadas.

Herramientas utilizadas

El proyecto se desarrolló utilizando el lenguaje de programación Python y el entorno Jupyter Notebook, que permitió organizar el procedimiento experimental en etapas de carga de datos, análisis, preprocesamiento, entrenamiento, validación y visualización de resultados. Para la manipulación de los conjuntos de datos se utilizó la biblioteca Pandas, mientras que NumPy permitió realizar operaciones numéricas y cálculos sobre arreglos. La generación de gráficas se realizó principalmente mediante Matplotlib, y se incorporó NetworkX para visualizar la estructura de dependencias de la Red Bayesiana.

La biblioteca Scikit-learn constituyó la herramienta principal para el desarrollo de los modelos de aprendizaje automático. La separación de los datos se realizó mediante train_test_split, mientras que la evaluación y selección de configuraciones se apoyó en KFold, StratifiedKFold, cross_validate, cross_val_predict y GridSearchCV. Estas herramientas permitieron mantener procedimientos de validación reproducibles y realizar la selección de hiperparámetros dentro del conjunto de entrenamiento.

Para el preprocesamiento se utilizaron Pipeline y ColumnTransformer, junto con SimpleImputer, OneHotEncoder, StandardScaler y KBinsDiscretizer, según las características de los datos y los requerimientos de cada algoritmo. Adicionalmente, en las redes neuronales de regresión se utilizó TransformedTargetRegressor, con el propósito de estandarizar la variable objetivo durante el entrenamiento y devolver las predicciones a su escala original.

Los algoritmos incorporados en esta segunda entrega fueron BayesianRidge para regresión bayesiana; GaussianNB y CategoricalNB para clasificación mediante Naive Bayes; SVR, LinearSVR, SVC y LinearSVC para Máquinas de Soporte Vectorial; y MLPRegressor y MLPClassifier para Redes Neuronales Artificiales. Asimismo, se recuperaron los modelos de

Árbol de Decisión, Random Forest y KNN de la primera entrega, junto con los modelos de referencia DummyRegressor y DummyClassifier, para establecer comparaciones sobre conjuntos de prueba comunes.

Para implementar la Red Bayesiana se utilizó la biblioteca pgmpy, específicamente HillClimbSearch para aprender la estructura probabilística, DiscreteBayesianNetwork para representar el grafo, BayesianEstimator para estimar las distribuciones condicionales y VariableElimination para realizar consultas de inferencia probabilística.

Adicionalmente, en el problema de clasificación se utilizó el módulo auxiliar helpers.py, desarrollado para centralizar funciones de preprocesamiento, cálculo de métricas y transformación de variables. Este módulo incorpora herramientas personalizadas para agrupar categorías poco frecuentes y tratar valores extremos, facilitando la reutilización de procedimientos y reduciendo la duplicación de código.

Finalmente, las métricas de desempeño se calcularon mediante las funciones de evaluación de Scikit-learn, mientras que los resultados se organizaron en tablas y representaciones gráficas para facilitar la comparación entre algoritmos, el análisis de su comportamiento y la interpretación de los experimentos realizados.

Métricas de evaluación

Para evaluar el desempeño de los algoritmos implementados se utilizaron métricas específicas para los problemas de regresión y clasificación, manteniendo los criterios establecidos en la primera entrega. La evaluación consideró tanto la capacidad predictiva de los modelos como su comportamiento durante la validación cruzada y sobre el conjunto TEST. Adicionalmente, se incorporaron medidas orientadas a analizar la complejidad de las redes neuronales y las dependencias entre variables.

Métricas de regresión

En el problema de regresión se utilizaron el Error Absoluto Medio (MAE), el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el coeficiente de determinación (R^2).

El MAE representa el promedio de las diferencias absolutas entre los valores reales y las

predicciones, permitiendo interpretar directamente la magnitud del error en milisegundos. El MSE calcula el promedio de los errores elevados al cuadrado, penalizando con mayor intensidad las predicciones que presentan desviaciones elevadas y expresándose en milisegundos cuadrados (ms^2). Por su parte, el RMSE corresponde a la raíz cuadrada del MSE, conserva las unidades originales de la variable objetivo y permite evaluar el error otorgando mayor peso a las desviaciones grandes.

El coeficiente de determinación R^2 mide el desempeño del modelo en relación con una predicción constante basada en el promedio de los valores reales del conjunto evaluado. Un valor de R^2 igual a 1 representa un ajuste perfecto, mientras que un valor de 0 corresponde al desempeño de dicha referencia. Los valores negativos indican que el modelo presenta un error cuadrático mayor que el de esa predicción constante. Para seleccionar y comparar los modelos de regresión se priorizó el RMSE, considerando también MAE y R^2 como indicadores complementarios.

Métricas de clasificación

En el problema de clasificación binaria se utilizaron Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y Average Precision. Se consideró la cancelación de una reserva como clase positiva (1) y la no cancelación como clase negativa (0).

Accuracy representa la proporción total de observaciones correctamente clasificadas. Sin embargo, debido a que aproximadamente el 62,96 % de las reservas pertenece a la clase no cancelada y el 37,04 % a la clase cancelada, esta métrica puede favorecer interpretaciones incompletas del desempeño. Por esta razón, se utilizó Balanced Accuracy, que calcula el promedio de las tasas de identificación correcta de ambas clases, otorgándoles la misma importancia independientemente de su frecuencia.

Precision expresa la proporción de reservas clasificadas como canceladas que efectivamente pertenecen a esa categoría. Recall, también denominado sensibilidad, representa la proporción de cancelaciones reales identificadas correctamente por el modelo. Como medida complementaria se consideró la especificidad (*Specificity*), que expresa la proporción de reservas no canceladas identificadas correctamente.

El F1-score corresponde a la media armónica entre Precision y Recall, permitiendo evaluar el equilibrio entre ambas métricas. Se estableció como criterio principal de comparación

y selección de modelos de clasificación, debido a que permite considerar conjuntamente los falsos positivos y los falsos negativos relacionados con la detección de cancelaciones.

Adicionalmente, se utilizaron ROC-AUC y Average Precision para evaluar la capacidad discriminativa de los clasificadores sin depender de un único umbral de decisión. ROC-AUC resume la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos a lo largo de diferentes umbrales. Por su parte, Average Precision (AP) sintetiza el comportamiento de Precision y Recall a partir de los valores obtenidos al variar el umbral, resultando especialmente útil para analizar el desempeño sobre la clase positiva. En las tablas del proyecto, esta medida se presenta bajo la denominación PR-AUC; sin embargo, su cálculo corresponde específicamente a average_precision_score de Scikit-learn y no a una integración trapezoidal de la curva Precision-Recall.

Métricas complementarias

Para analizar la complejidad de las redes neuronales se calculó el número de conexiones entre neuronas. Esta medida corresponde a la cantidad de pesos asociados a las conexiones entre capas consecutivas, sin incluir los términos de sesgo (*bias*). Su cálculo considera el número de neuronas de entrada, de cada capa oculta y de la capa de salida. De esta manera, se utilizó como indicador del tamaño de las arquitecturas y permitió comparar el incremento de complejidad frente a las variaciones del desempeño predictivo.

Por otra parte, para examinar el supuesto de independencia condicional de Naive Bayes se utilizó la V de Cramér, una medida de asociación entre variables categóricas basada en el estadístico chi-cuadrado. Sus valores se encuentran entre 0 y 1, donde los valores cercanos a cero indican una asociación débil o inexistente en los datos analizados, mientras que los valores próximos a uno representan una asociación más fuerte. En este experimento, la medida se calculó dentro de cada clase y se resumió mediante un promedio ponderado, permitiendo identificar dependencias entre los atributos seleccionados. Esta medida describe asociaciones estadísticas, pero no demuestra relaciones causales ni constituye por sí sola una prueba exhaustiva de independencia.

Finalmente, en los procedimientos de validación cruzada se reportaron las medias y desviaciones estándar de las métricas entre los cinco folds. Estas medidas permitieron describir el desempeño promedio y su variabilidad entre particiones, facilitando la comparación de las

configuraciones evaluadas. No obstante, la desviación estándar entre folds se interpretó como una medida descriptiva y no como una prueba formal de significancia estadística.

RESULTADOS

Regresión

Punto de partida: modelos de la primera entrega

Como punto de partida para la evaluación de los algoritmos incorporados en la segunda entrega, se recuperaron y entrenaron nuevamente los modelos de regresión desarrollados previamente: Árbol de Decisión podado, Random Forest podado y K-Nearest Neighbors (KNN), junto con el modelo de referencia basado en la media (DummyRegressor). Para este procedimiento se conservaron las configuraciones finales de hiperparámetros, el preprocesamiento correspondiente a cada algoritmo y la partición Hold-Out 70/30 utilizada en la primera entrega, manteniendo la semilla aleatoria random_state=42.

El objetivo de esta verificación fue comprobar la reproducibilidad de los resultados anteriores y establecer una referencia experimental para comparar los nuevos modelos bajo un conjunto común de observaciones de prueba. Como se presenta en la Tabla 2, los valores de RMSE obtenidos nuevamente coinciden, con la precisión numérica reportada, con los registrados en la primera entrega. Esta correspondencia respalda la consistencia de la partición y del procedimiento de evaluación utilizado, permitiendo comparar el desempeño predictivo de los algoritmos de ambas entregas sobre el mismo conjunto TEST.

Modelo	**RMSE TEST (ms)**	**R^2 TEST**	**RMSE reportado en la Entrega 1**	**¿Coincide?**
Árbol podado CV	15.3323	0.3502	15.3323	Sí
Random Forest podado CV	15.1495	0.3656	15.1495	Sí
KNN K = 50, distance	15.2923	0.3536	15.2923	Sí
Baseline, media	19.0207	0.0000	19.0207	Sí

Tabla 2. Verificación de los modelos de la primera entrega sobre TEST (regresión).

Resultados de la regresión bayesiana

La regresión lineal bayesiana presentó un desempeño predictivo favorable en el problema de estimación del tiempo de reacción. Durante la validación cruzada de cinco particiones, realizada sobre una muestra reproducible de 8.000 registros de TRAIN, obtuvo un RMSE medio

de 15,0483 ± 0,3281 ms y un coeficiente de determinación R^2 de 0,3886. Posteriormente, el modelo entrenado con los 175.000 registros del conjunto TRAIN alcanzó sobre TEST un MAE de 12,0130 ms, un RMSE de 15,0609 ms y un R^2 de 0,3730. Estos resultados representan una mejora respecto al Random Forest podado de la primera entrega, cuyo RMSE fue de 15,1495 ms. No obstante, la diferencia de aproximadamente 0,089 ms es reducida, por lo que debe interpretarse considerando también la variabilidad observada durante la validación cruzada.

Adicionalmente, el modelo estimó una desviación estándar del ruido de aproximadamente 15,07 ms, valor cercano al RMSE obtenido sobre TEST. Esta correspondencia resulta compatible con la estimación probabilística del error realizada por BayesianRidge, aunque no permite afirmar que toda la variabilidad restante sea irreducible o que no pueda ser explicada mediante otras variables o modelos.

Una de las principales ventajas de la regresión bayesiana es la posibilidad de interpretar los coeficientes estimados y cuantificar su incertidumbre. Debido al preprocesamiento utilizado, los coeficientes de las variables numéricas estandarizadas indican el cambio esperado en el tiempo de reacción, expresado en milisegundos, asociado con un incremento de una desviación estándar en el atributo correspondiente, manteniendo constantes las demás variables del modelo. En cambio, los coeficientes de las variables categóricas codificadas mediante One-Hot Encoding requieren una interpretación diferente, relacionada con su representación mediante indicadores. La Tabla 3 presenta los coeficientes estimados y sus desviaciones estándar posteriores, mientras que la Figura 9 muestra gráficamente sus magnitudes y los intervalos de credibilidad aproximados del 95 %.

Variable	Coeficiente (ms por desviación estándar)	Incertidumbre (± 1 desviación)
Sleep_Hours	−7.471	0.037
PlayTime_Hours	7.166	0.046
Age	5.720	0.036
Energy_Drinks	−0.354	0.042
Game_Genre_Battle Royale	0.106	1.712
Blue_Light_Filter_Yes	0.092	0.075
Game_Genre_FPS	−0.086	1.710
Game_Genre_Fighting	−0.052	1.713
HRV_Score	0.041	0.046
Game_Genre_MOBA	0.033	1.710
Screen_Brightness	−0.019	0.036
Heart_Rate_BPM	0.008	0.044
APM	−0.005	0.067

Tabla 3. Coeficientes de la regresión bayesiana y su incertidumbre.

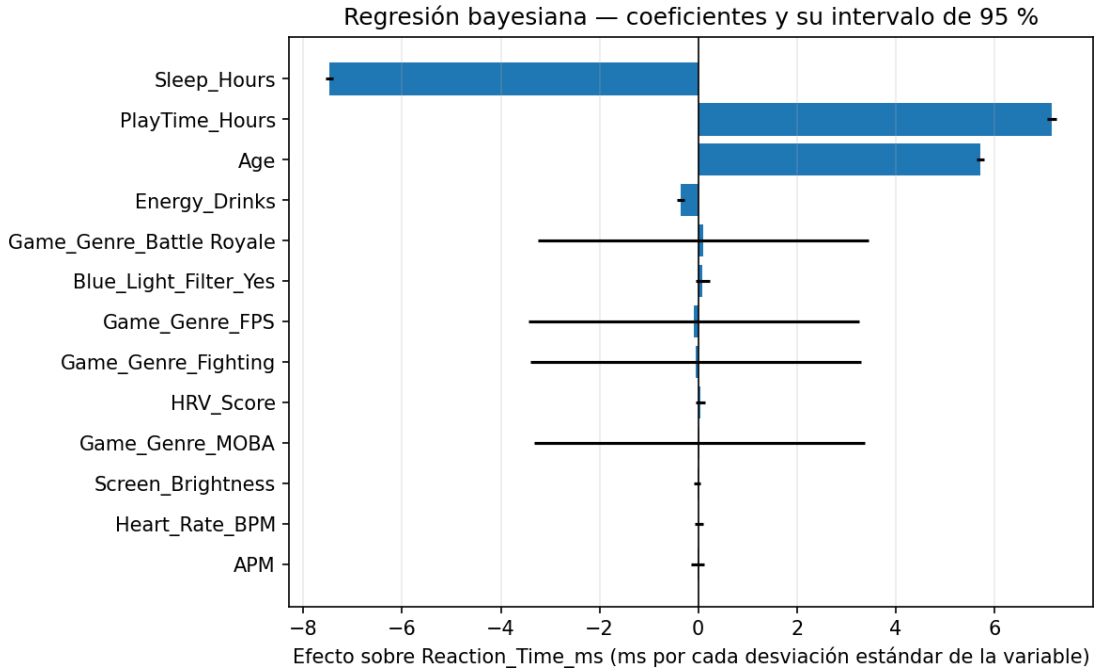


Figura 9. Coeficientes de la regresión bayesiana con su intervalo de 95 %.

El análisis de los coeficientes muestra que Sleep_Hours, PlayTime_Hours y Age presentan las mayores magnitudes dentro de las variables numéricas estandarizadas. En particular, Sleep_Hours obtuvo un coeficiente de −7,471, lo que indica que un incremento de una desviación estándar en las horas de sueño se asocia, según el modelo, con una disminución esperada de aproximadamente 7,47 ms en el tiempo de reacción, manteniendo constantes las demás características. Por su parte, PlayTime_Hours presentó un coeficiente de 7,166 y Age uno de 5,720, indicando asociaciones positivas con el tiempo de reacción. Estas relaciones corresponden a estimaciones estadísticas condicionadas por las variables incluidas en el modelo y no deben interpretarse como efectos causales demostrados.

Los demás atributos presentaron coeficientes de menor magnitud, aunque su interpretación debe considerar tanto el escalado como la incertidumbre asociada. Un resultado particularmente interesante corresponde a HRV_Score, cuya correlación lineal individual con Reaction_Time_ms fue de aproximadamente −0,295 en la primera entrega, mientras que su coeficiente en la regresión bayesiana fue cercano a cero (0,041). Esta diferencia indica que la asociación observada de manera individual disminuye considerablemente cuando el modelo incorpora simultáneamente las demás variables. El resultado puede estar relacionado con información compartida entre atributos o con las características del ajuste multivariable, pero no demuestra por sí solo que HRV_Score carezca de valor predictivo en otros modelos o configuraciones.

En conjunto, los resultados son coherentes con los análisis realizados en la primera entrega, donde las horas de sueño, el tiempo dedicado al juego y la edad mostraron una capacidad predictiva individual relevante. Además, las incertidumbres de los coeficientes permiten distinguir entre estimaciones relativamente precisas y otras menos definidas, especialmente en algunas variables categóricas. Esto evidencia la utilidad de la regresión bayesiana para complementar el desempeño predictivo con herramientas de interpretación probabilística.

Además de estimar el tiempo de reacción esperado, BayesianRidge permite calcular una desviación estándar predictiva para cada observación mediante return_std=True. A partir de esta

estimación se construyeron intervalos predictivos aproximados del 95 %, definidos como el valor predicho ± 1,96 veces su desviación estándar. A diferencia de los intervalos presentados en la Figura 9, que describen la incertidumbre de los coeficientes del modelo, estos intervalos buscan representar la variabilidad esperada del valor de una nueva observación, considerando los supuestos probabilísticos de BayesianRidge. La Tabla 4 presenta ocho ejemplos seleccionados del conjunto TEST, incluyendo el tiempo real, la predicción, el margen de incertidumbre y la verificación de pertenencia al intervalo correspondiente.

Jugador	**Real (ms)**	**Predicho (ms)**	**Intervalo 95 % (ms)**	**Error (ms)**	**¿Real dentro del intervalo?**
1	230.83	256.39	± 29.72	−25.56	Sí
2	283.01	265.84	± 29.72	17.17	Sí
3	251.81	270.49	± 29.72	−18.68	Sí
4	253.91	275.42	± 29.72	−21.51	Sí
5	258.00	273.91	± 29.72	−15.91	Sí
6	284.60	274.34	± 29.72	10.26	Sí
7	277.94	270.35	± 29.72	7.59	Sí
8	226.66	266.19	± 29.72	−39.53	No

Tabla 4. Ejemplos de predicción con intervalo de incertidumbre (regresión bayesiana).

En los ejemplos presentados se observa que siete de las ocho mediciones reales se encuentran dentro de sus respectivos intervalos predictivos, mientras que una observación presenta una desviación superior al margen estimado. Al extender esta evaluación a los 75.000 registros del conjunto TEST, se obtuvo una cobertura empírica del 95,1 %, muy cercana al nivel nominal del 95 %. Este resultado indica que, para el conjunto de prueba evaluado, los intervalos generados por BayesianRidge presentan una cobertura global compatible con la esperada bajo sus supuestos probabilísticos. Sin embargo, esta correspondencia no garantiza una calibración igualmente adecuada para todos los grupos de observaciones ni para conjuntos de datos futuros. En consecuencia, los intervalos deben interpretarse como estimaciones de incertidumbre condicionadas por el modelo y por las características de los datos utilizados.

Resultados de la máquina de soporte vectorial

La selección de hiperparámetros de la Máquina de Soporte Vectorial para regresión se realizó mediante GridSearchCV, evaluando 18 configuraciones correspondientes a las combinaciones de dos kernels (linear y rbf), tres valores de regularización (C = 1, 10 y 100) y tres valores de tolerancia (ε = 0,5; 2 y 5). Cada configuración se evaluó mediante validación cruzada de cinco particiones sobre una muestra reproducible de 8.000 registros de TRAIN, utilizando el RMSE como criterio principal de selección. La Tabla 5 presenta los resultados ordenados de menor a mayor error de validación cruzada.

Los resultados muestran que las nueve configuraciones con kernel lineal ocuparon los primeros puestos, con valores de RMSE comprendidos aproximadamente entre 15,120 y 15,125 ms. La mejor configuración correspondió a kernel='linear', C=1 y epsilon=5, alcanzando un RMSE medio de 15,1201 ms y un R^2 medio de 0,3752. Las diferencias entre las configuraciones lineales fueron reducidas, lo que indica que, dentro del rango evaluado, el desempeño presentó poca sensibilidad a los valores seleccionados de C y epsilon.

Por otra parte, la mejor configuración con kernel RBF obtuvo un RMSE de 15,3369 ms, utilizando C=10 y epsilon=5. Al incrementar C hasta 100, el RMSE de validación cruzada aumentó, alcanzando valores entre 15,9965 y 16,1208 ms. Este comportamiento muestra que una penalización más elevada de los errores no necesariamente mejora la capacidad predictiva y puede afectar negativamente la generalización. Aunque los resultados son compatibles con un posible incremento del sobreajuste, no permiten confirmarlo de manera concluyente sin analizar también el desempeño sobre los datos de entrenamiento.

Kernel	C	ε	RMSE CV (ms)	Desv. RMSE CV	R² CV
lineal	1	5.0	15.1201	0.2363	0.3752
lineal	10	5.0	15.1203	0.2362	0.3752
lineal	100	5.0	15.1203	0.2361	0.3752
lineal	1	0.5	15.1214	0.2401	0.3751
lineal	100	0.5	15.1220	0.2399	0.3750
lineal	10	0.5	15.1222	0.2401	0.3750
lineal	1	2.0	15.1239	0.2411	0.3749
lineal	10	2.0	15.1252	0.2412	0.3748
lineal	100	2.0	15.1253	0.2413	0.3748
RBF	10	5.0	15.3369	0.2215	0.3572
RBF	10	2.0	15.3655	0.2216	0.3548
RBF	10	0.5	15.3717	0.2159	0.3543
RBF	1	2.0	15.3798	0.2180	0.3537
RBF	1	0.5	15.3810	0.2201	0.3536
RBF	1	5.0	15.4006	0.2122	0.3520
RBF	100	5.0	15.9965	0.2616	0.3008
RBF	100	2.0	16.0642	0.2535	0.2949
RBF	100	0.5	16.1208	0.2609	0.2899

Tabla 5. Configuraciones de la SVR evaluadas mediante validación cruzada.

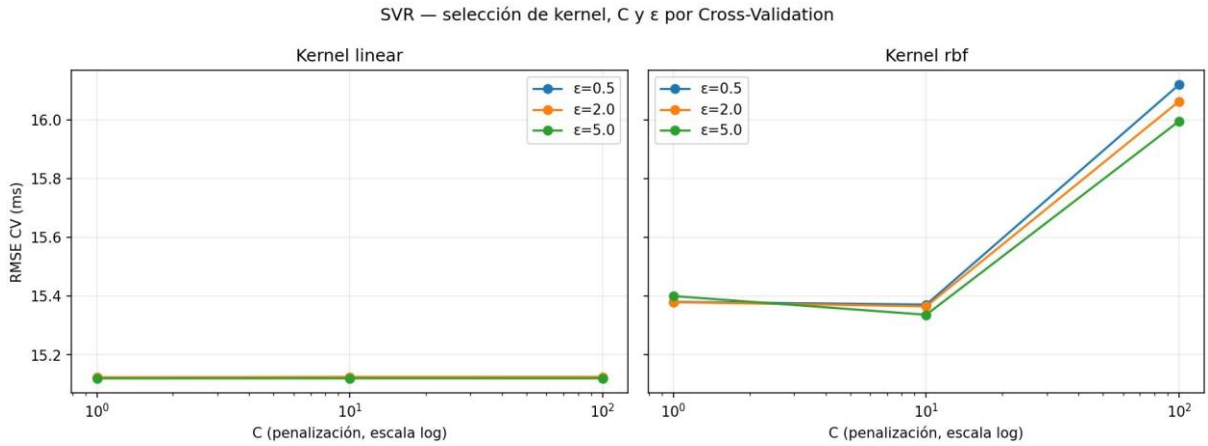


Figura 10. Selección de kernel, C y ε de la SVR mediante validación cruzada.

Como se observa en la Figura 10, el kernel lineal presentó un comportamiento más estable frente a los cambios de los hiperparámetros evaluados, mientras que el kernel RBF mostró una mayor sensibilidad al incremento de C, especialmente cuando este alcanzó el valor de 100. Dentro de las configuraciones analizadas, el kernel lineal obtuvo los menores errores de

validación cruzada. Este resultado sugiere que una representación lineal de las relaciones entre las variables predictoras y el tiempo de reacción resulta competitiva para este conjunto de datos. Sin embargo, no demuestra que el fenómeno sea estrictamente lineal ni descarta que otras configuraciones no lineales puedan ofrecer un mejor desempeño.

A partir de los resultados de GridSearchCV, se seleccionó como referencia la configuración lineal con C=1 y epsilon=5. Para el entrenamiento final se utilizó LinearSVR, debido a que su implementación permite trabajar de manera más eficiente con conjuntos de datos de gran tamaño. A diferencia de la búsqueda inicial, realizada con SVR(kernel='linear') sobre una muestra de 8.000 registros, el modelo final se entrenó utilizando los 175.000 registros disponibles en TRAIN. Es importante señalar que ambos estimadores utilizan formulaciones de optimización diferentes, por lo que la configuración seleccionada inicialmente constituye una referencia experimental y no garantiza que los hiperparámetros sean óptimos para LinearSVR.

Durante la evaluación final sobre los 75.000 registros de TEST, el modelo LinearSVR obtuvo un RMSE de 15,0617 ms y un R^2 de 0,3729. Estos resultados superaron ligeramente el desempeño del Random Forest podado de la primera entrega, cuyo RMSE fue de 15,1495 ms, y fueron prácticamente equivalentes a los de la regresión lineal bayesiana, que alcanzó un RMSE de 15,0609 ms. La diferencia entre ambos modelos lineales fue inferior a 0,001 ms, lo que evidencia un comportamiento predictivo muy similar sobre el conjunto de prueba evaluado.

La comparación entre el RMSE de validación cruzada de SVR(kernel='linear') y el RMSE en TEST de LinearSVR debe interpretarse con precaución, debido a que corresponden a estimadores diferentes, entrenados con cantidades distintas de observaciones y evaluados sobre conjuntos diferentes. Por tanto, la diferencia entre sus resultados no puede atribuirse exclusivamente al aumento del tamaño de entrenamiento. En conjunto, los hallazgos muestran que el enfoque lineal constituye una alternativa competitiva para este problema de regresión, con un desempeño cercano al de la regresión bayesiana y un costo computacional manejable.

Resultados de las tres redes neuronales

Para analizar el efecto de la complejidad arquitectónica sobre el desempeño predictivo se evaluaron tres redes neuronales de regresión mediante MLPRegressor, utilizando las arquitecturas suave (16), intermedia (64, 64) y grande (128, 128, 128). Las tres configuraciones se evaluaron mediante validación cruzada de cinco particiones sobre una muestra reproducible de 20.000 registros de TRAIN y posteriormente se entrenaron con los 175.000 registros completos para su evaluación final sobre TEST. La Tabla 6 presenta los resultados obtenidos, incluyendo el número de capas ocultas, las neuronas por capa, las conexiones, el RMSE de validación cruzada y las métricas finales de predicción.

Los resultados muestran un comportamiento predictivo muy similar entre las tres arquitecturas. La red suave obtuvo un RMSE medio de validación cruzada de 15,131 ms, mientras que las redes intermedia y grande alcanzaron 15,132 y 15,133 ms, respectivamente. Estas diferencias, del orden de milésimas de milisegundo, son considerablemente menores que las desviaciones estándar observadas entre los folds, comprendidas entre 0,082 y 0,099 ms. Por tanto, dentro de las condiciones evaluadas, no se identificaron mejoras relevantes del desempeño promedio de validación al incrementar simultáneamente la profundidad y el número de neuronas.

En el conjunto TEST se observó un comportamiento semejante. La red grande presentó el menor RMSE, con 14,9956 ms, seguida por la red suave, con 14,9960 ms, y la intermedia, con 14,9983 ms. Aunque la red grande ocupó el primer lugar en esta evaluación, su ventaja frente a la red suave fue de aproximadamente 0,0005 ms, una diferencia de magnitud muy reducida. Por tanto, no resulta suficiente para afirmar que una arquitectura más compleja ofrece una mejora predictiva relevante. Asimismo, las tres redes obtuvieron valores de R^2 cercanos a 0,378, lo que indica que presentaron una capacidad explicativa prácticamente igual sobre el conjunto de prueba.

Red	**Capas ocultas**	**Neuronas por capa**	**Conexiones**	**RMSE CV (ms)**	**RMSE TEST (ms)**	**R^2 TEST**	**Respecto a la red anterior**
Suave (16)	1	16	224	15.131 ± 0.090	14.996	0.3784	Punto de partida

Intermedia (64, 64)	2	64	4.992	15.132 ± 0.082	14.998	0.3782	Quedó igual (+0.001)
Grande (128, 128, 128)	3	128	34.560	15.133 ± 0.099	14.996	0.3784	Quedó igual (+0.001)

Tabla 6. Comparación de las tres redes neuronales de regresión.

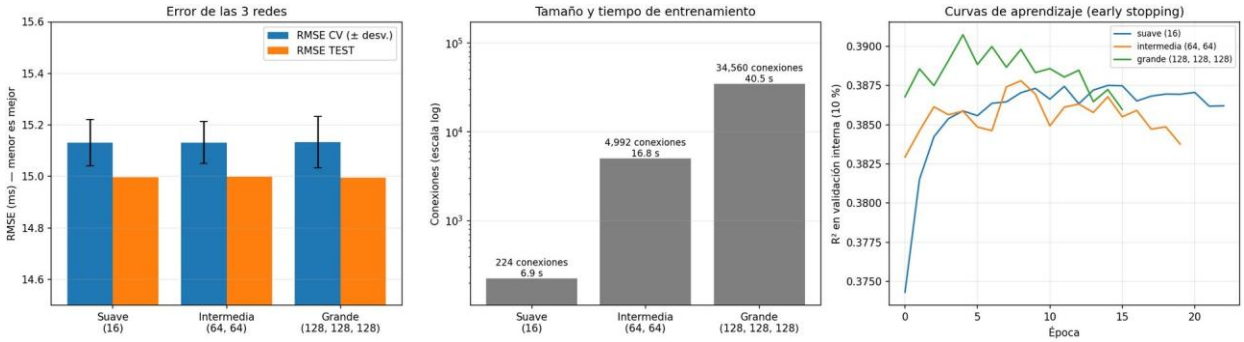


Figura 11. Error, tamaño, tiempo de entrenamiento y curvas de aprendizaje de las tres redes.

La Figura 11 permite comparar el desempeño predictivo de las tres arquitecturas con su complejidad y costo computacional. Mientras que la red suave contiene 224 conexiones, la intermedia presenta 4.992 y la grande alcanza 34.560, lo que representa un incremento superior a 154 veces entre la primera y la última arquitectura. Este aumento también se refleja en los tiempos de entrenamiento registrados: aproximadamente 7,0 segundos para la red suave, 17,3 segundos para la intermedia y 40,3 segundos para la grande. En consecuencia, la arquitectura de mayor tamaño requirió aproximadamente 5,8 veces el tiempo de entrenamiento de la más sencilla, sin conseguir una reducción relevante del error predictivo.

Las curvas presentadas en la tercera gráfica de la Figura 11 corresponden a la evolución del coeficiente R^2 sobre la partición de validación interna utilizada por el mecanismo de detención temprana (*early stopping*). Se observa que la red suave completó 23 épocas, la intermedia 20 y la grande 16. Estas diferencias muestran que las arquitecturas alcanzaron sus respectivos criterios de detención en cantidades distintas de iteraciones. Sin embargo, completar menos épocas no implica necesariamente una mayor eficiencia computacional, debido a que cada época de una red más grande puede requerir un mayor número de operaciones.

En conjunto, los resultados muestran que el incremento de la complejidad arquitectónica no produjo una mejora relevante del RMSE dentro de las configuraciones estudiadas. Aunque las diferencias observadas son inferiores a la variabilidad entre folds, este criterio constituye una

comparación descriptiva y no una prueba formal de equivalencia estadística. Por tanto, considerando conjuntamente el error de validación cruzada, la cantidad de conexiones y el tiempo de entrenamiento, la red suave representa la alternativa más eficiente entre las tres arquitecturas principales evaluadas.

Para complementar este análisis, se evaluaron nueve combinaciones de arquitectura, incluidas las tres redes principales, combinando una, dos y tres capas ocultas con 16, 64 y 128 neuronas por capa. Este procedimiento permitió estudiar por separado el efecto de aumentar la cantidad de capas y el número de neuronas. La Tabla 7 presenta los resultados de estas combinaciones, así como las diferencias observadas al modificar cada característica arquitectónica.

Arquitectura	**Conexiones**	**RMSE CV (ms)**	**Al agregar una capa**	**Al aumentar neuronas**
(16)	224	15.131 ± 0.090	—	—
(64)	896	15.127 ± 0.087	—	Quedó igual (−0.004)
(128)	1.792	15.127 ± 0.047	—	Quedó igual (−0.000)
(16, 16)	480	15.176 ± 0.109	Quedó igual (+0.045)	—
(64, 64)	4.992	15.132 ± 0.082	Quedó igual (+0.005)	Quedó igual (−0.044)
(128, 128)	18.176	15.152 ± 0.087	Quedó igual (+0.025)	Quedó igual (+0.020)
(16, 16, 16)	736	15.160 ± 0.075	Quedó igual (−0.015)	—
(64, 64, 64)	9.088	15.159 ± 0.088	Quedó igual (+0.027)	Quedó igual (−0.001)
(128, 128, 128)	34.560	15.133 ± 0.099	Quedó igual (−0.019)	Quedó igual (−0.026)

Tabla 7. Efecto de agregar capas y neuronas en las nueve combinaciones (regresión).

Como se observa en la Figura 12, los valores de RMSE de validación cruzada de las nueve arquitecturas se concentran en un intervalo reducido, comprendido aproximadamente entre 15,127 y 15,176 ms. La configuración de una capa oculta con 128 neuronas presentó el menor error medio (15,1268 ms), seguida muy de cerca por la arquitectura de una capa con 64 neuronas (15,1268 ms) y la red suave de 16 neuronas (15,1312 ms). No obstante, las diferencias entre estas configuraciones fueron considerablemente menores que las desviaciones estándar registradas durante la validación cruzada. Asimismo, las arquitecturas con dos y tres capas ocultas no mostraron una reducción consistente del error frente a las configuraciones de una sola capa. Estos resultados sugieren que, para el conjunto de datos y las condiciones experimentales consideradas, aumentar la profundidad o la cantidad de neuronas no proporcionó beneficios predictivos relevantes. En consecuencia, los resultados favorecen la utilización de arquitecturas menos complejas cuando ofrecen un desempeño comparable, considerando su menor cantidad de conexiones y los requerimientos computacionales asociados.

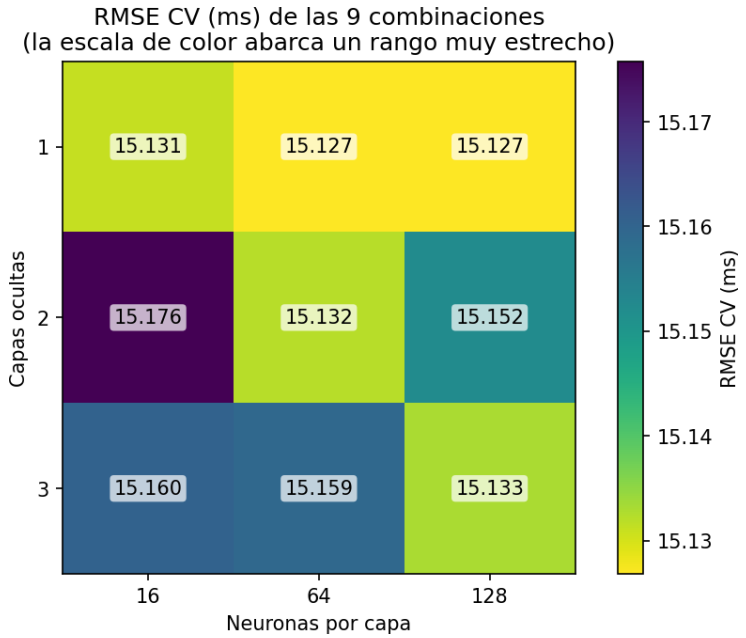


Figura 12. RMSE CV de las nueve combinaciones de capas y neuronas.

Comparación final de los modelos

Evaluación sobre el mismo conjunto TEST

Con el propósito de comparar el desempeño predictivo de los algoritmos desarrollados en ambas entregas, se realizó una evaluación conjunta utilizando el mismo conjunto TEST, compuesto por 75.000 registros que no participaron en el entrenamiento de los modelos. Esta comparación incluyó los tres algoritmos supervisados de la primera entrega, las cinco configuraciones incorporadas en la segunda y el modelo de referencia basado en la media. Para garantizar la consistencia de la evaluación se utilizaron las mismas métricas: MAE, MSE, RMSE y R^2, priorizando el RMSE como criterio principal de comparación. Adicionalmente, se calculó el porcentaje de reducción del RMSE respecto al baseline, permitiendo cuantificar la mejora obtenida por cada algoritmo frente a una estrategia de predicción constante. La Tabla 8 presenta los resultados ordenados de menor a mayor RMSE.

Modelo	**Entrega**	**MAE**	**MSE**	**RMSE**	**R^2**	**Reducción del RMSE frente al baseline (%)**
Red neuronal grande	E2	11.9533	224.8669	14.9956	0.3784	21.16
Red neuronal suave	E2	11.9592	224.8811	14.9960	0.3784	21.16
Red neuronal intermedia	E2	11.9553	224.9486	14.9983	0.3782	21.15
Regresión bayesiana	E2	12.0130	226.8301	15.0609	0.3730	20.82
SVR lineal (C = 1, ε = 5)	E2	12.0135	226.8563	15.0617	0.3729	20.81
Random Forest podado CV	E1	12.0862	229.5079	15.1495	0.3656	20.35
KNN K = 50, distance	E1	12.1874	233.8547	15.2923	0.3536	19.60
Árbol podado CV	E1	12.2279	235.0786	15.3323	0.3502	19.39
Baseline, media	—	15.1553	361.7885	19.0207	−0.0000	0.00

Tabla 8. Comparación final de los modelos de ambas entregas sobre el mismo conjunto TEST (regresión).

Los resultados de la Tabla 8 muestran que todas las configuraciones incorporadas en la segunda entrega obtuvieron menores errores de predicción que los algoritmos evaluados en la primera. Las tres redes neuronales ocuparon los primeros puestos, con valores de RMSE cercanos a 15,00 ms y coeficientes de determinación alrededor de 0,378. La red neuronal grande presentó el menor RMSE (14,9956 ms), seguida por la red suave (14,9960 ms) y la intermedia (14,9983 ms). Sin embargo, las diferencias entre estas arquitecturas fueron inferiores a 0,003 ms,

por lo que no constituyen una mejora relevante desde el punto de vista práctico.

La regresión lineal bayesiana y la SVR lineal obtuvieron resultados prácticamente equivalentes, con RMSE de 15,0609 y 15,0617 ms, respectivamente. Ambos modelos superaron ligeramente al Random Forest podado, que había obtenido el mejor desempeño en la primera entrega, con un RMSE de 15,1495 ms. Por su parte, KNN y el Árbol de Decisión podado conservaron las posiciones siguientes, alcanzando errores de 15,2923 y 15,3323 ms. Estos resultados muestran que los modelos incorporados en la segunda fase lograron representar patrones predictivos que permitieron reducir el error frente a los enfoques previamente evaluados.

Al comparar la mejor configuración de la segunda entrega con Random Forest, se observa una reducción absoluta del RMSE de aproximadamente 0,154 ms, equivalente a una mejora relativa cercana al 1,02 % respecto al error de este último. Por tanto, aunque el orden de los algoritmos favorece a las redes neuronales, las diferencias deben interpretarse considerando su magnitud y los requerimientos computacionales de cada modelo. En particular, la red suave consiguió prácticamente el mismo desempeño que la red grande utilizando una arquitectura considerablemente más sencilla y un menor tiempo de entrenamiento.

Respecto al modelo de referencia, cuyo RMSE fue de 19,0207 ms, los algoritmos evaluados lograron reducciones del error comprendidas entre aproximadamente el 19,39 % y el 21,16 %. La red neuronal grande alcanzó la mayor reducción, con un 21,16 %. No obstante, los coeficientes de determinación, inferiores a 0,38 en todas las configuraciones, evidencian que una proporción considerable de la variabilidad del tiempo de reacción permanece sin explicar por los modelos. Esto sugiere que las características disponibles contienen información predictiva relevante, pero no permiten representar completamente las diferencias observadas entre los jugadores.

En consecuencia, el menor RMSE sobre TEST identifica a la red grande como la configuración con mejor resultado puntual dentro de la evaluación realizada, pero no justifica por sí solo seleccionarla como la opción definitiva. Para establecer una recomendación deben considerarse también el desempeño mediante validación cruzada, la estabilidad del algoritmo, la complejidad arquitectónica y el costo computacional. La Figura 13 presenta la comparación visual del RMSE y del coeficiente R^2 de los modelos evaluados.

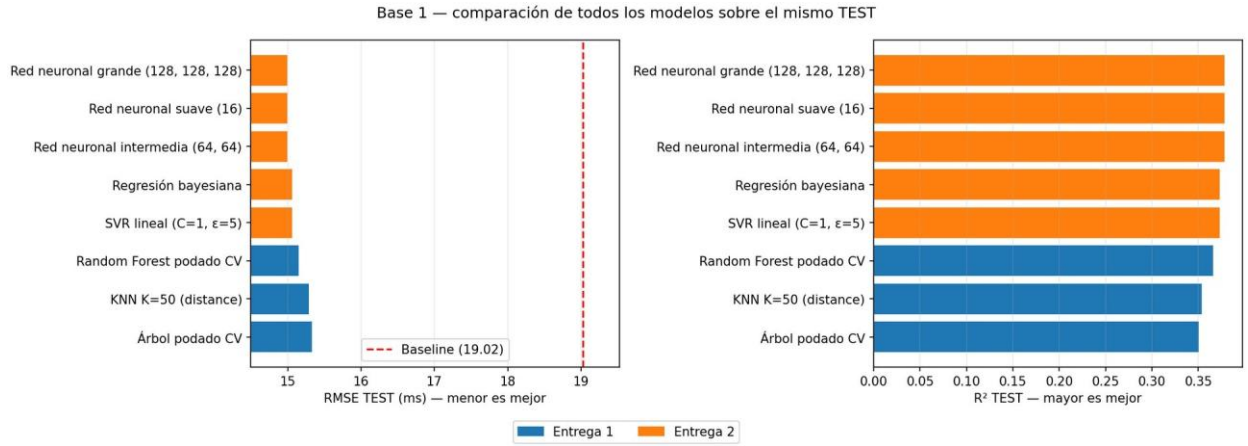


Figura 13. Comparación del RMSE (escala ampliada) y del R^2 de los modelos sobre TEST.

Comparación común mediante Cross-Validation

Para complementar la comparación realizada sobre TEST y analizar la estabilidad del desempeño entre diferentes particiones de entrenamiento y validación, se evaluaron los modelos finales mediante un procedimiento común de validación cruzada. Este experimento utilizó una muestra reproducible de 8.000 registros del conjunto TRAIN y las mismas cinco particiones para todos los algoritmos, manteniendo las configuraciones previamente establecidas. Se calcularon el RMSE medio, su desviación estándar, el MAE medio y el coeficiente R^2 medio. Este procedimiento permite comparar los modelos bajo condiciones comunes de evaluación, aunque no constituye una validación completamente independiente de los procesos anteriores de selección de configuraciones. La Tabla 9 presenta los resultados obtenidos.

Modelo	**RMSE CV, media ± desviación**	**MAE CV medio**	**R^2 CV medio**
Regresión bayesiana	15.0483 ± 0.3281	12.0093	0.3886
SVR lineal (C = 1, ε = 5)	15.0768 ± 0.3288	12.0268	0.3863
Red neuronal intermedia	15.1062 ± 0.3681	12.0312	0.3838
Red neuronal suave	15.1326 ± 0.3340	12.0581	0.3817
Red neuronal grande	15.1401 ± 0.3453	12.0612	0.3811
Random Forest podado CV	15.3361 ± 0.3344	12.2590	0.3650
KNN K = 50, distance	15.6981 ± 0.2967	12.4916	0.3346
Árbol podado CV	15.8501 ± 0.3087	12.6504	0.3216
Baseline, media	19.2487 ± 0.3250	15.3944	−0.0005

Tabla 9. Comparación común mediante Cross-Validation (regresión).

Los resultados de la Tabla 9 muestran que los cinco modelos incorporados en la segunda entrega obtuvieron un RMSE medio de validación cruzada inferior al de los algoritmos desarrollados anteriormente. La regresión lineal bayesiana presentó el menor error promedio, con 15,0483 ± 0,3281 ms, seguida por la SVR lineal, con 15,0768 ± 0,3288 ms. Entre las redes neuronales, la arquitectura intermedia obtuvo el menor RMSE medio (15,1062 ms), mientras que las configuraciones suave y grande alcanzaron 15,1326 y 15,1401 ms, respectivamente. Random Forest se mantuvo como el mejor algoritmo de la primera entrega, con un RMSE medio de 15,3361 ms.

A diferencia de la evaluación sobre TEST, donde las tres redes neuronales ocuparon los primeros puestos, la validación cruzada favoreció ligeramente a la regresión bayesiana y a la SVR lineal. Esta variación del orden es compatible con las diferencias entre los conjuntos utilizados para entrenamiento y evaluación, la sensibilidad de los algoritmos a las muestras y la variabilidad propia del procedimiento experimental. Además, las diferencias observadas entre los modelos nuevos fueron pequeñas respecto a las desviaciones estándar registradas entre folds, por lo que no permiten establecer una superioridad estadística concluyente de una configuración sobre las demás.

La comparación conjunta indica que los nuevos algoritmos ofrecieron un desempeño predictivo competitivo y, dentro de las condiciones evaluadas, superior al de los modelos anteriores. Sin embargo, los resultados también muestran que incrementar la complejidad no garantiza obtener mejoras relevantes. En este sentido, la regresión bayesiana destaca por su desempeño y capacidad de interpretación probabilística, mientras que la red neuronal suave constituye una alternativa favorable cuando se consideran conjuntamente la precisión predictiva y la simplicidad arquitectónica. La elección final del modelo debe responder al equilibrio entre desempeño, estabilidad, interpretabilidad y costo computacional, evitando fundamentarse exclusivamente en una posición dentro de las tablas de resultados.

Datos reales frente a predicciones

La figura 14 compara el tiempo de reacción real de 4.000 jugadores de TEST con el predicho por cada modelo. Si un modelo fuera perfecto, todos los puntos caerían sobre la diagonal roja.

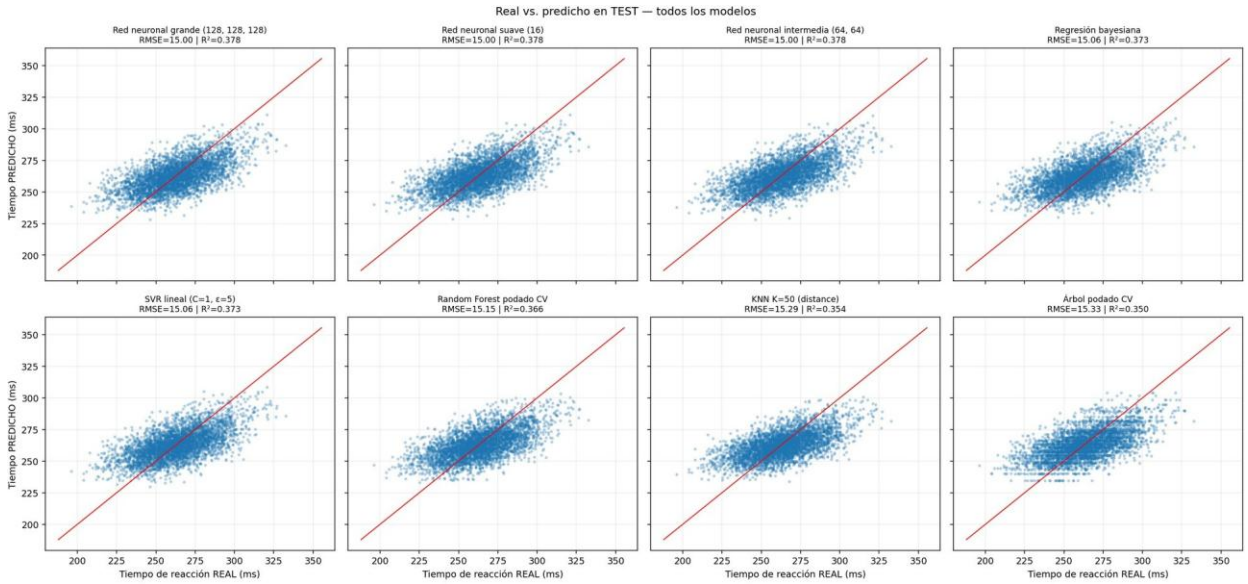


Figura 14. Valores reales frente a valores predichos en TEST para todos los modelos.

La Figura 14 permite examinar visualmente la relación entre los tiempos de reacción reales y las predicciones generadas por los modelos. La línea diagonal representa la condición de predicción perfecta, en la cual el valor estimado coincide exactamente con el observado. Los puntos ubicados por encima de esta línea corresponden a predicciones superiores al valor real, mientras que los situados por debajo representan subestimaciones. La dispersión alrededor de la diagonal permite identificar la magnitud y el comportamiento general de los errores.

Se observa que los modelos concentran sus predicciones en un intervalo más estrecho que el de los valores reales. En términos generales, los tiempos de reacción más bajos tienden a ser sobreestimados, mientras que los más elevados tienden a ser subestimados. Este comportamiento indica que los algoritmos presentan dificultades para representar completamente los valores extremos y producen estimaciones relativamente cercanas a los valores centrales de la distribución. Este patrón es coherente con los coeficientes R^2 moderados obtenidos durante la evaluación.

Asimismo, las predicciones del Árbol de Decisión presentan agrupamientos horizontales característicos de su estructura, debido a que cada hoja asigna un valor constante a las

observaciones que cumplen determinadas condiciones. Por el contrario, la regresión bayesiana, la SVR lineal y las redes neuronales producen distribuciones de predicciones visualmente más continuas. Sin embargo, estas diferencias en la representación gráfica no permiten concluir que la relación real entre los atributos y el tiempo de reacción sea exclusivamente lineal.

En conjunto, la Figura 14 complementa los resultados cuantitativos al evidenciar que, aunque los modelos identifican asociaciones predictivas relevantes, todavía presentan errores y limitaciones para representar toda la variabilidad del tiempo de reacción. Por tanto, las mejoras obtenidas durante la segunda entrega deben interpretarse dentro del alcance de los datos y las configuraciones experimentales evaluadas, sin asumir que los algoritmos han alcanzado necesariamente el máximo desempeño posible para este problema.

Clasificación

Punto de partida: modelos de la primera entrega

Como punto de partida para evaluar los nuevos algoritmos de clasificación, se recuperaron y entrenaron nuevamente los tres modelos desarrollados en la primera entrega: Árbol de Decisión podado, Random Forest ajustado y K-Nearest Neighbors (KNN). Para garantizar la continuidad del procedimiento experimental, se conservaron sus configuraciones finales de hiperparámetros, los respectivos pipelines de preprocesamiento y los umbrales de clasificación previamente seleccionados mediante validación cruzada.

Los modelos se entrenaron utilizando los 83.573 registros de TRAIN y se evaluaron sobre los mismos 35.817 registros de TEST empleados en la primera entrega, manteniendo la partición Hold-Out estratificada 70/30 y la semilla aleatoria random_state=42. Este procedimiento tuvo como finalidad verificar la reproducibilidad de los resultados anteriores y establecer una referencia común para comparar el desempeño de los algoritmos incorporados en la segunda entrega.

Como se presenta en la Tabla 10, los valores de F1-score obtenidos coincidieron con los reportados previamente, mientras que los resultados de ROC-AUC conservaron sus valores originales dentro de la precisión numérica utilizada. Esta correspondencia respalda la consistencia de la implementación y del procedimiento de evaluación, aunque no constituye por sí sola una garantía de ausencia de errores metodológicos. La reproducción de estos resultados permite establecer una comparación experimental entre ambas entregas, evitando introducir cambios innecesarios en las configuraciones de los modelos anteriores.

Modelo	**Umbral**	**F1 TEST**	**ROC-AUC TEST**	**F1 reportado en la Entrega 1**	**¿Coincide?**
Random Forest ajustado	0.49	0.7955	0.9288	0.7955	Sí
Árbol podado	0.58	0.7708	0.9128	0.7708	Sí
KNN ajustado	0.43	0.7592	0.9014	0.7592	Sí

Tabla 10. Verificación de los modelos de la primera entrega sobre TEST (clasificación).

Entre los modelos recuperados, Random Forest ajustado presentó el mejor desempeño predictivo, alcanzando un F1-score de 0,7955 y un ROC-AUC de 0,9288, utilizando un umbral de clasificación de 0,49. Estos resultados indican que el modelo consiguió un equilibrio favorable entre la precisión de las predicciones positivas y la capacidad para identificar las reservas realmente canceladas, además de presentar una elevada capacidad de discriminación entre ambas clases a lo largo de diferentes umbrales.

Por su parte, el Árbol de Decisión podado obtuvo un F1-score de 0,7708 y un ROC-AUC de aproximadamente 0,9128, mientras que KNN alcanzó valores de 0,7592 y 0,9014, respectivamente. Las diferencias de F1 respecto a Random Forest fueron de aproximadamente 0,0246 para el Árbol podado y 0,0363 para KNN. Aunque los tres algoritmos mostraron una capacidad predictiva favorable, Random Forest presentó los mejores resultados entre las configuraciones evaluadas en la primera entrega.

La ventaja observada de Random Forest es compatible con las características de los métodos de ensamble, que combinan múltiples árboles de decisión para reducir la variabilidad asociada a un único estimador. Sin embargo, estos resultados no permiten atribuir exclusivamente a dicho mecanismo las diferencias de desempeño, debido a que los algoritmos presentan distintas capacidades de representación, estrategias de regularización y configuraciones de entrenamiento.

Es importante señalar que los umbrales de clasificación se conservaron sin realizar nuevos ajustes sobre TEST. Esta decisión permitió mantener los criterios establecidos mediante validación cruzada durante la primera entrega y evitar que los resultados del conjunto de prueba influyeran en la selección de las configuraciones recuperadas.

En consecuencia, Random Forest ajustado se establece como el principal referente de clasificación para la segunda entrega. Los nuevos algoritmos serán evaluados considerando principalmente el F1-score, complementado con Balanced Accuracy, Precision, Recall, ROC-AUC y Average Precision, con el propósito de determinar si ofrecen mejoras en la identificación de cancelaciones o ventajas relacionadas con la interpretabilidad, la estabilidad y el costo computacional.

Resultados de Naive Bayes

Antes de evaluar el desempeño predictivo de Naive Bayes, se realizó un análisis exploratorio para examinar su principal supuesto probabilístico: la independencia condicional entre los atributos dada la clase. Para ello, se utilizó la V de Cramér, una medida de asociación entre variables categóricas basada en el estadístico chi-cuadrado. El análisis se desarrolló sobre once atributos seleccionados y discretizados del conjunto TRAIN, calculando las asociaciones entre pares de variables por separado para las reservas canceladas y no canceladas.

Posteriormente, los valores de asociación de ambas clases se combinaron mediante un promedio ponderado, utilizando como pesos las proporciones de registros correspondientes a cada categoría. De esta manera, se obtuvo una medida descriptiva de la asociación entre atributos considerando la distribución de las clases. La Figura 15 presenta estos resultados mediante una matriz de calor, mientras que la Tabla 11 identifica los ocho pares de variables con mayores valores de asociación.

Los resultados muestran que determinadas características de las reservas mantienen asociaciones estadísticas incluso cuando se analizan dentro de una misma clase. Esto evidencia que el supuesto de independencia condicional no se satisface completamente para los atributos estudiados. No obstante, la V de Cramér únicamente permite describir la intensidad de las asociaciones categóricas y no demuestra relaciones causales ni permite evaluar exhaustivamente todas las posibles dependencias entre las variables originales del conjunto de datos.

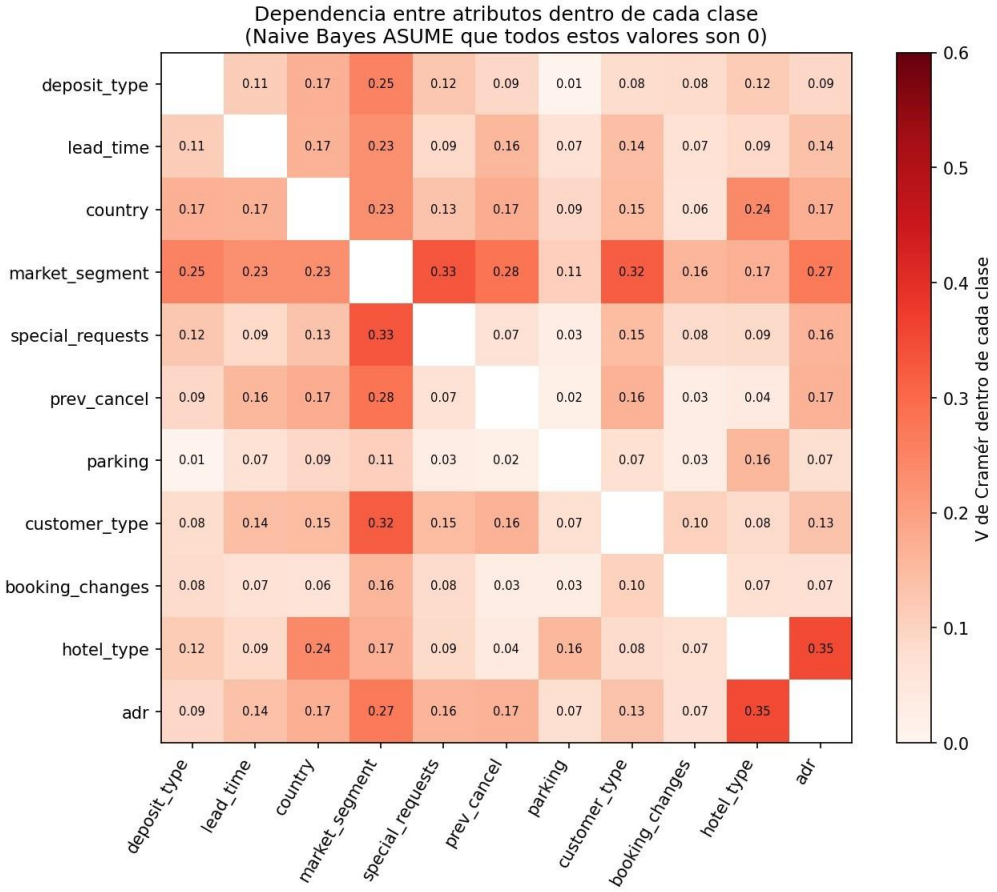


Figura 15. Dependencia entre atributos dentro de cada clase (V de Cramér).

Atributo A	**Atributo B**	**V de Cramér dentro de cada clase**
hotel_type	adr	0.351
market_segment	special_requests	0.332
market_segment	customer_type	0.319
market_segment	prev_cancel	0.279
market_segment	adr	0.268
deposit_type	market_segment	0.252
country	hotel_type	0.241
country	market_segment	0.229

Tabla 11. Pares de atributos más dependientes entre sí dentro de cada clase.

Entre las asociaciones más relevantes se identificó la relación entre el tipo de hotel (hotel_type) y la tarifa de alojamiento (adr), con una V de Cramér de 0,351. También se encontraron asociaciones entre el segmento de mercado (market_segment) y las solicitudes especiales (special_requests), con 0,332, así como entre el segmento de mercado y el tipo de cliente (customer_type), con 0,319. Otras relaciones relevantes involucraron el historial de cancelaciones, el tipo de depósito y el país de procedencia. Estos resultados indican que algunas características de las reservas contienen información estadísticamente relacionada incluso dentro de las clases de cancelación. En consecuencia, la suposición de independencia condicional de Naive Bayes constituye una simplificación de las relaciones presentes en los datos, cuya influencia sobre el desempeño predictivo debe examinarse mediante la evaluación de los modelos y su posterior comparación con la Red Bayesiana.

La Tabla 12 presenta los resultados de las ocho configuraciones de Naive Bayes evaluadas mediante validación cruzada estratificada de cinco particiones sobre el conjunto TRAIN. Se analizaron cuatro configuraciones de GaussianNB, modificando el parámetro var_smoothing, y cuatro de CategoricalNB, combinando diferentes cantidades de intervalos de discretización (n_bins) y valores de suavizado (alpha). La comparación permitió examinar cómo los supuestos probabilísticos y las estrategias de representación de los atributos influyen sobre el desempeño de clasificación.

Entre las configuraciones categóricas, el mejor resultado correspondió a CategoricalNB con diez intervalos y alpha=0,5, que alcanzó un ROC-AUC medio de 0,8315, un F1-score de 0,6436, una Precision de 0,8002 y un Recall de 0,5383. Por su parte, el mayor ROC-AUC obtenido por GaussianNB fue de 0,8090, correspondiente a var_smoothing=0,1. Sin embargo, la configuración gaussiana con var_smoothing=10^{-6} presentó un F1-score de 0,6543, ligeramente superior al de la mejor variante categórica, aunque con un ROC-AUC inferior de 0,8017.

Estas diferencias evidencian que el orden de los modelos depende de la métrica utilizada. Mientras que F1-score evalúa el equilibrio entre Precision y Recall para un umbral específico, ROC-AUC resume la capacidad del clasificador para discriminar entre ambas clases a través de distintos puntos de corte. Debido a que posteriormente se ajustaría el umbral de decisión para

maximizar F1, se utilizó ROC-AUC como criterio principal para seleccionar la variante, priorizando su capacidad discriminativa antes del ajuste del punto de corte.

En consecuencia, se seleccionó CategoricalNB con n_bins=10 y alpha=0,5, al presentar el mayor ROC-AUC medio entre las configuraciones evaluadas. Esta selección corresponde al mejor resultado dentro del espacio de búsqueda considerado y no implica que CategoricalNB sea necesariamente superior a GaussianNB bajo cualquier configuración o conjunto de datos.

Variante	**Parámetros**	**F1 CV**	**Recall CV**	**Precision CV**	**Balanced Acc. CV**	**ROC-AUC CV**
CategoricalNB	n_bins = 10, alpha = 0.5	0.6436	0.5383	0.8002	0.7296	0.8315
CategoricalNB	n_bins = 10, alpha = 1.0	0.6426	0.5360	0.8022	0.7291	0.8314
CategoricalNB	n_bins = 5, alpha = 0.5	0.6323	0.5226	0.8001	0.7229	0.8260
CategoricalNB	n_bins = 5, alpha = 1.0	0.6313	0.5201	0.8028	0.7225	0.8260
GaussianNB	var_smoothing = 0.1	0.6201	0.8799	0.4787	0.6580	0.8090
GaussianNB	var_smoothing = 0.001	0.6421	0.7966	0.5379	0.6969	0.8024
GaussianNB	var_smoothing = 1e-09	0.6000	0.5063	0.7824	0.7034	0.8017
GaussianNB	var_smoothing = 1e-06	0.6543	0.7304	0.5951	0.7171	0.8017

Tabla 12. Variantes de Naive Bayes evaluadas mediante validación cruzada.

Una vez seleccionada la variante categórica, se realizó el ajuste del umbral de clasificación utilizando probabilidades fuera de muestra (*out-of-fold*) obtenidas mediante validación cruzada sobre TRAIN. El procedimiento evaluó diferentes puntos de corte y seleccionó el valor que maximizó el F1-score, obteniendo un umbral final de 0,28. Este resultado indica que, para las condiciones experimentales consideradas, un punto de decisión inferior al valor predeterminado de 0,5 permitió establecer un equilibrio más favorable entre la identificación de cancelaciones y la precisión de las predicciones positivas.

La Figura 16 presenta la distribución de las probabilidades de cancelación estimadas por Naive Bayes sobre TEST, diferenciando las reservas realmente canceladas de las no canceladas. Se observan concentraciones de predicciones cercanas a cero y a uno, acompañadas de una zona

de superposición entre ambas clases. Esta distribución evidencia que el modelo produce numerosas estimaciones probabilísticas extremas, comportamiento compatible con los efectos que puede generar la suposición de independencia condicional cuando existen atributos relacionados.

Sin embargo, la presencia de probabilidades extremas no demuestra por sí sola que el modelo esté mal calibrado ni permite atribuir exclusivamente este comportamiento a las dependencias identificadas mediante la V de Cramér. Para evaluar formalmente la calibración probabilística sería necesario incorporar procedimientos específicos, como curvas de calibración o el Brier Score. En este experimento, la Figura 16 se utiliza como recurso descriptivo para interpretar la distribución de las predicciones y la ubicación del umbral seleccionado.

La evaluación final se realizó sobre los 35.817 registros del conjunto TEST, utilizando el umbral de 0,28 establecido previamente. Los resultados se presentan en la Tabla 13.

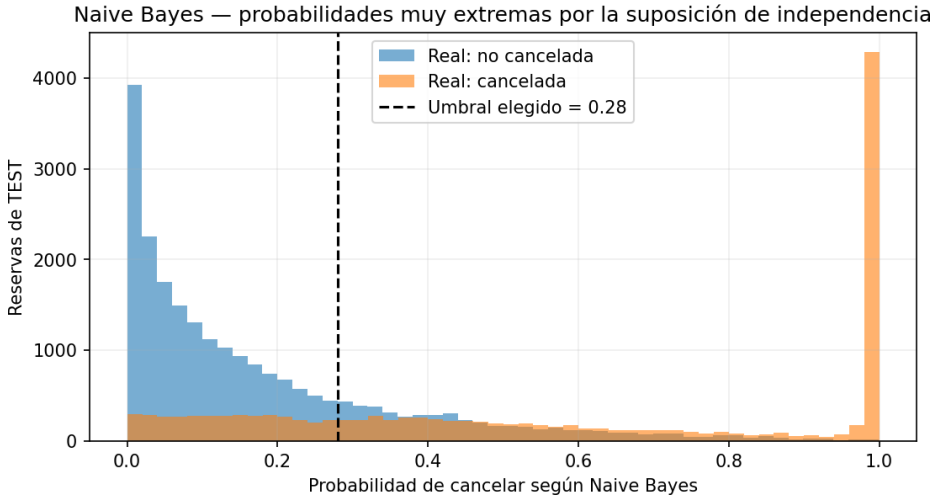


Figura 16. Distribución de las probabilidades de cancelación estimadas por Naive Bayes.

Métrica	**Resultado**
Umbral	0.28
Accuracy	0.7579
Balanced Accuracy	0.7505
Precision	0.6579
Recall	0.7219
F1-score	0.6884
ROC-AUC	0.8308
PR-AUC	0.7957

Tabla 13. Resultados TEST de Naive Bayes categórico.

Los resultados de la Tabla 13 muestran que el modelo CategoricalNB, utilizando el umbral de clasificación de 0,28, obtuvo una Accuracy de 0,7579 y una Balanced Accuracy de 0,7505. Estos valores indican que el clasificador logró identificar correctamente una proporción considerable de las reservas, aunque su capacidad para reconocer ambas clases de manera equilibrada fue inferior a la observada en los modelos con mejor desempeño de la primera entrega.

En relación con la detección de cancelaciones, el modelo alcanzó una Precision de 0,6579 y un Recall de 0,7219. Esto significa que aproximadamente el 65,79 % de las reservas clasificadas como canceladas efectivamente pertenecía a esa categoría, mientras que el modelo consiguió identificar cerca del 72,19 % de las cancelaciones reales. La combinación de ambas métricas produjo un F1-score de 0,6884, lo que evidencia un desempeño predictivo útil, aunque con una proporción relevante de falsos positivos y cancelaciones no detectadas.

Por otra parte, el ROC-AUC de 0,8308 muestra que el modelo conserva capacidad para discriminar entre reservas canceladas y no canceladas considerando diferentes umbrales de decisión. Asimismo, se obtuvo un valor de Average Precision de 0,7957, presentado como PR-AUC en las tablas del proyecto. Estas métricas complementan el análisis del F1-score y permiten observar que, aunque el clasificador presenta limitaciones en la asignación de etiquetas, sus puntuaciones contienen información relevante para diferenciar las clases.

Al comparar estos resultados con los modelos de la primera entrega, Naive Bayes presentó el menor F1-score y ROC-AUC entre las configuraciones supervisadas evaluadas hasta este punto. En particular, Random Forest obtuvo un F1-score de 0,7955, superior en aproximadamente 0,1071 al valor alcanzado por Naive Bayes. Sin embargo, el modelo bayesiano superó al clasificador de referencia basado en la clase mayoritaria, que presentó una Accuracy aproximada de 0,6296 y un F1-score de cero para la clase cancelada.

En términos computacionales, Naive Bayes destacó por su rapidez de entrenamiento y predicción, lo que constituye una ventaja cuando se requiere procesar grandes volúmenes de información con recursos limitados. No obstante, dentro de las condiciones experimentales estudiadas, esta eficiencia no compensó completamente su menor desempeño predictivo frente a los demás algoritmos.

En conjunto, los resultados muestran que Naive Bayes representa una alternativa

probabilística sencilla y computacionalmente eficiente, pero con limitaciones para capturar las dependencias entre las características de las reservas hoteleras. El análisis mediante la V de Cramér identificó asociaciones entre determinadas variables, lo que proporciona una posible explicación parcial de las limitaciones observadas, sin demostrar que estas dependencias sean la única causa del menor desempeño. Este hallazgo motiva la comparación posterior con una Red Bayesiana, capaz de incorporar explícitamente relaciones probabilísticas entre atributos mediante una estructura aprendida a partir de los datos.

Resultados de la Red Bayesiana

La Red Bayesiana permitió representar las dependencias probabilísticas existentes entre las características seleccionadas de las reservas hoteleras y la variable objetivo is_canceled. Mediante el algoritmo HillClimbSearch, utilizando el criterio de información BIC y una restricción máxima de tres padres por nodo, se obtuvo una estructura compuesta por doce variables y treinta conexiones dirigidas. El grafo resultante se presenta en la Figura 17, donde las flechas representan las relaciones de dependencia utilizadas para factorizar la distribución conjunta de las variables.

El análisis de la estructura aprendida mostró que is_canceled tiene como nodos padres a deposit_type, market_segment y country, lo que indica que estas variables intervienen directamente en la distribución condicional de la cancelación dentro del modelo. Asimismo, se identificaron conexiones dirigidas desde is_canceled hacia lead_time, special_requests, parking, booking_changes y prev_cancel. Estas relaciones describen la estructura estadística seleccionada por el algoritmo y no deben interpretarse como relaciones causales, debido a que el aprendizaje se realizó a partir de datos observacionales y no mediante intervenciones experimentales.

Adicionalmente, se analizó el manto de Markov (*Markov blanket*) de is_canceled, conformado por sus nodos padres, sus hijos y los demás padres de sus hijos. Este conjunto representa las variables que contienen la información necesaria para determinar la distribución condicional de la clase cuando se conocen los valores de las demás características del modelo. En la estructura obtenida, adr, correspondiente a la tarifa de alojamiento, fue la única variable que quedó fuera del manto de Markov. Esto significa que, bajo las independencias condicionales

representadas por la red, adr no aporta información adicional sobre la cancelación una vez conocidos los valores del manto de Markov. Sin embargo, no implica que la tarifa carezca de relación estadística con la cancelación o que no pueda resultar útil en otros modelos.

En conjunto, la estructura aprendida muestra que la información relacionada con las condiciones económicas, el segmento de mercado, el país de procedencia y las características de la reserva participa en diferentes relaciones probabilísticas. Esta representación permite estudiar asociaciones entre variables y constituye una alternativa más flexible que la estructura de independencia condicional asumida por Naive Bayes, aunque las conexiones obtenidas dependen de los datos, la discretización y las restricciones del procedimiento de aprendizaje.

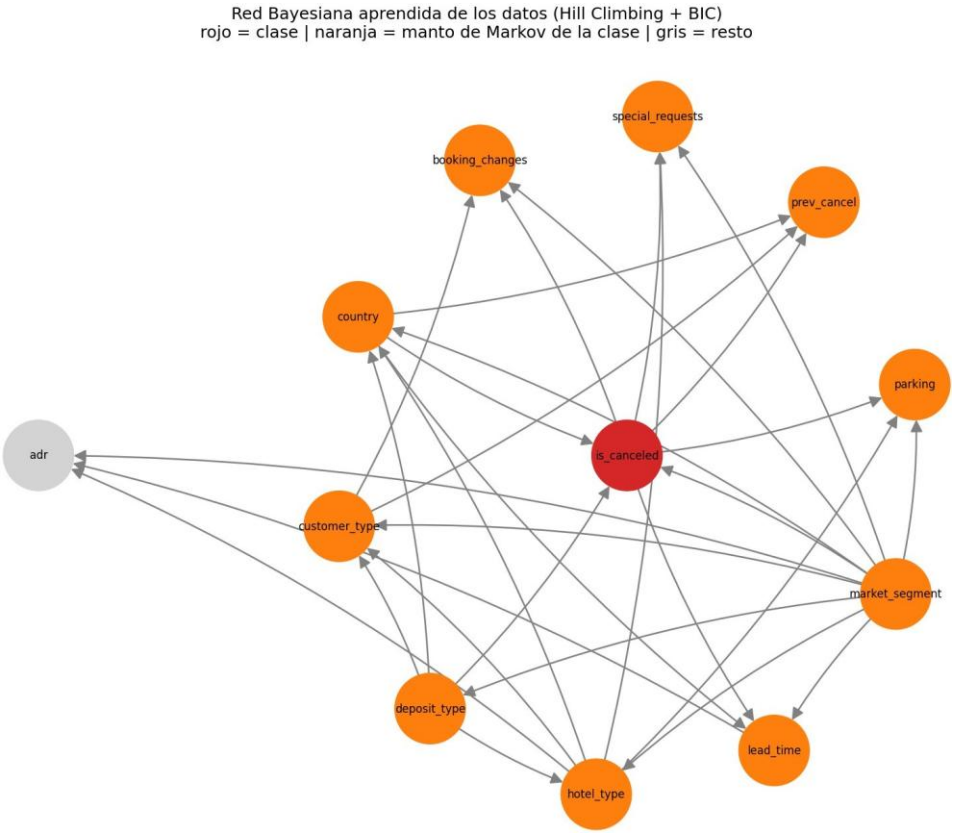


Figura 17. Red Bayesiana aprendida a partir de los datos.

Una de las características de las Redes Bayesianas es su capacidad para realizar inferencia probabilística a partir de la estructura y las distribuciones condicionales aprendidas. Para aprovechar esta propiedad, se utilizó el algoritmo VariableElimination de la biblioteca pgmpy, que permite calcular la distribución de probabilidad de una variable de interés considerando diferentes combinaciones de evidencia conocida.

En este experimento se realizaron consultas sobre la probabilidad estimada de cancelación, condicionándola a determinadas características de las reservas hoteleras. Se evaluaron situaciones relacionadas con el tipo de depósito, la anticipación de la reserva, las solicitudes especiales, los espacios de estacionamiento, los antecedentes de cancelación y la combinación de segmento de mercado y país de procedencia. La Tabla 14 presenta los resultados obtenidos para estas condiciones.

Estas consultas permiten observar cómo varía la probabilidad de cancelación estimada por el modelo cuando se introduce información adicional sobre una reserva. Sin embargo, los resultados corresponden a probabilidades condicionales generadas por la Red Bayesiana y no deben interpretarse como efectos causales de las características consultadas. Asimismo, las estimaciones dependen de la estructura aprendida, la representación discreta de los atributos y las distribuciones de probabilidad calculadas durante el entrenamiento.

Evidencia conocida sobre la reserva	**P(cancelar)**
Sin información (probabilidad base)	37.0 %
Depósito no reembolsable	99.4 %
Sin depósito	28.3 %
Reservó con más de 180 días de anticipación	56.9 %
Pidió estacionamiento	0.0 %
Hizo 2 o más solicitudes especiales	20.8 %
Canceló antes otra reserva	91.9 %
Agencia online + Portugal	45.4 %

Tabla 14. Consultas de probabilidad sobre la Red Bayesiana.

Los resultados de la Tabla 14 muestran diferencias importantes entre las probabilidades de cancelación estimadas bajo las condiciones consultadas. Sin información adicional sobre la reserva, el modelo presenta una probabilidad de cancelación de aproximadamente 37,0 %, coherente con la proporción de reservas canceladas en el conjunto de datos. Al incorporar la condición de depósito no reembolsable, la probabilidad estimada aumenta a 99,4 %, mientras que para las reservas sin depósito disminuye a 28,3 %. Estos resultados indican que el tipo de depósito proporciona información relevante para diferenciar los perfiles de cancelación dentro del modelo.

El historial de cancelaciones también presentó una asociación importante: cuando se considera que el cliente ha cancelado previamente otra reserva, la probabilidad estimada alcanza el 91,9 %. Por su parte, una anticipación superior a 180 días se relacionó con una probabilidad del 56,9 %, mientras que realizar dos o más solicitudes especiales presentó una probabilidad estimada del 20,8 %. La condición de haber solicitado estacionamiento produjo una probabilidad cercana al 0,05 %. Estas diferencias muestran que el modelo identifica perfiles de reserva asociados con diferentes niveles de probabilidad de cancelación.

La consulta conjunta correspondiente a una reserva realizada mediante una agencia de viajes en línea y asociada con Portugal produjo una probabilidad de cancelación del 45,4 %. Este resultado ilustra la posibilidad de incorporar simultáneamente varias características como evidencia para obtener estimaciones condicionadas a diferentes perfiles de reserva.

Es importante señalar que estos valores no representan efectos causales ni garantizan que las probabilidades observadas se reproduzcan en poblaciones diferentes. En particular, las estimaciones muy cercanas a cero o a uno deben interpretarse considerando la cantidad de observaciones disponibles para cada combinación de características y los supuestos probabilísticos del modelo. Por tanto, las consultas permiten explorar asociaciones y generar interpretaciones sobre la información aprendida, pero no sustituyen una evaluación específica de calibración o de incertidumbre de las probabilidades estimadas.

Para analizar el desempeño de la Red Bayesiana frente a un enfoque probabilístico más sencillo, se realizó una comparación con una variante de Naive Bayes entrenada sobre exactamente los mismos once atributos discretizados. Esta estrategia permitió establecer condiciones de comparación más homogéneas respecto a la información utilizada por los modelos, evitando que las diferencias de desempeño se atribuyeran únicamente a la inclusión de variables predictoras adicionales.

Naive Bayes utiliza una estructura fija basada en el supuesto de independencia condicional entre atributos dada la clase, mientras que la Red Bayesiana incorpora una estructura de dependencias aprendida mediante búsqueda heurística. No obstante, los modelos también presentan diferencias en sus procedimientos de estimación y suavizado de probabilidades, por lo que la comparación permite evaluar el desempeño conjunto de ambos enfoques, pero no aislar completamente el efecto de la estructura probabilística.

En ambos casos, los umbrales de clasificación se seleccionaron utilizando predicciones fuera de muestra obtenidas mediante validación cruzada dentro de TRAIN, mientras que la evaluación final se realizó sobre el mismo conjunto TEST. La Tabla 15 presenta los resultados comparativos de los dos modelos.

Modelo (mismos 11 atributos)	**Estructura**	**Umbral**	**F1**	**Recall**	**Precision**	**Balanced Acc.**	**ROC-AUC**
Naive Bayes	Fija: independencia supuesta	0.27	0.7011	0.7293	0.6749	0.7613	0.8495
Red Bayesiana	Aprendida: 30 dependencias	0.41	0.7531	0.7485	0.7578	0.8039	0.8993

Tabla 15. Naive Bayes frente a Red Bayesiana con los mismos once atributos (TEST).

Los resultados de la Tabla 15 muestran que la Red Bayesiana obtuvo un desempeño superior al de Naive Bayes cuando ambos modelos utilizaron los mismos once atributos discretizados. El F1-score aumentó de 0,7011 a 0,7531, lo que representa una diferencia absoluta de aproximadamente 0,0520. Asimismo, el ROC-AUC pasó de 0,8495 a 0,8993, con una mejora de 0,0498. Estas diferencias muestran que, dentro de las condiciones experimentales evaluadas, la Red Bayesiana logró un mejor equilibrio predictivo y una mayor capacidad discriminativa.

El análisis de las métricas complementarias permite observar que la mejora también se presentó en Precision, que aumentó de 0,6749 a 0,7578, y en Recall, que pasó de 0,7293 a 0,7485. Esto indica que la Red Bayesiana consiguió aumentar la proporción de cancelaciones correctamente identificadas y, simultáneamente, reducir la proporción relativa de falsos positivos entre las reservas clasificadas como canceladas. Por su parte, Balanced Accuracy aumentó de 0,7613 a 0,8039, evidenciando un mejor desempeño promedio en la identificación de ambas clases.

Estos hallazgos son compatibles con la hipótesis de que representar dependencias entre atributos puede resultar beneficioso frente a una estructura que supone independencia condicional. Sin embargo, la mejora observada no puede atribuirse exclusivamente a este factor, debido a que también existen diferencias en los procedimientos de estimación y suavizado de probabilidades. En consecuencia, los resultados permiten afirmar que la configuración de Red Bayesiana evaluada superó a la variante comparable de Naive Bayes, pero no cuantifican de

manera aislada el efecto de abandonar el supuesto de independencia.

Adicionalmente, la Red Bayesiana alcanzó un ROC-AUC de 0,8993 utilizando once atributos discretizados, valor cercano al 0,9014 obtenido por KNN en la primera entrega. Este comportamiento resulta relevante porque muestra que una representación probabilística con un subconjunto de variables puede alcanzar una capacidad discriminativa competitiva. No obstante, las diferencias de preprocesamiento, representación y entrenamiento deben considerarse al interpretar la comparación.

En conjunto, la Red Bayesiana constituye una alternativa de clasificación con un desempeño favorable y una ventaja interpretativa adicional: permite representar explícitamente relaciones probabilísticas entre atributos y formular consultas condicionadas a diferentes características de las reservas. Estas propiedades complementan las métricas de clasificación y permiten estudiar la estructura estadística aprendida por el modelo, sin asumir que las conexiones identificadas representan relaciones causales demostradas.

Resultados de la máquina de soporte vectorial

La tabla 16 presenta las doce configuraciones evaluadas. El kernel RBF con C = 1 y class_weight = 'balanced' obtuvo el mayor F1 de validación cruzada (0.7659), frente a 0.7375 del mejor kernel lineal. A diferencia de la regresión, donde el kernel lineal fue superior, en clasificación la frontera curva del kernel RBF separa mejor las clases. También se observa el efecto del tratamiento del desbalance: con el mismo kernel y C, class_weight = 'balanced' aumenta el Recall.

Kernel	C	class_weight	F1 CV	Desv. F1 CV	Recall CV	Precision CV	ROC-AUC CV
RBF	1	balanced	0.7659	0.0116	0.7746	0.7578	0.8941
RBF	10	None	0.7591	0.0080	0.7216	0.8009	0.8901
RBF	10	balanced	0.7525	0.0082	0.7692	0.7367	0.8903
RBF	1	None	0.7417	0.0157	0.6564	0.8524	0.8911
lineal	0.01	balanced	0.7375	0.0102	0.7567	0.7195	0.8824
lineal	0.1	balanced	0.7375	0.0097	0.7584	0.7179	0.8822
lineal	1	balanced	0.7373	0.0095	0.7597	0.7164	0.8819
RBF	0.1	balanced	0.7248	0.0127	0.7033	0.7485	0.8708
lineal	0.1	None	0.7129	0.0128	0.6277	0.8252	0.8818
lineal	1	None	0.7128	0.0129	0.6284	0.8238	0.8815
lineal	0.01	None	0.7120	0.0139	0.6217	0.8333	0.8821

RBF	0.1	None	0.6058	0.0203	0.4499	0.9282	0.8750

Tabla 16. Configuraciones de la SVM evaluadas mediante validación cruzada.

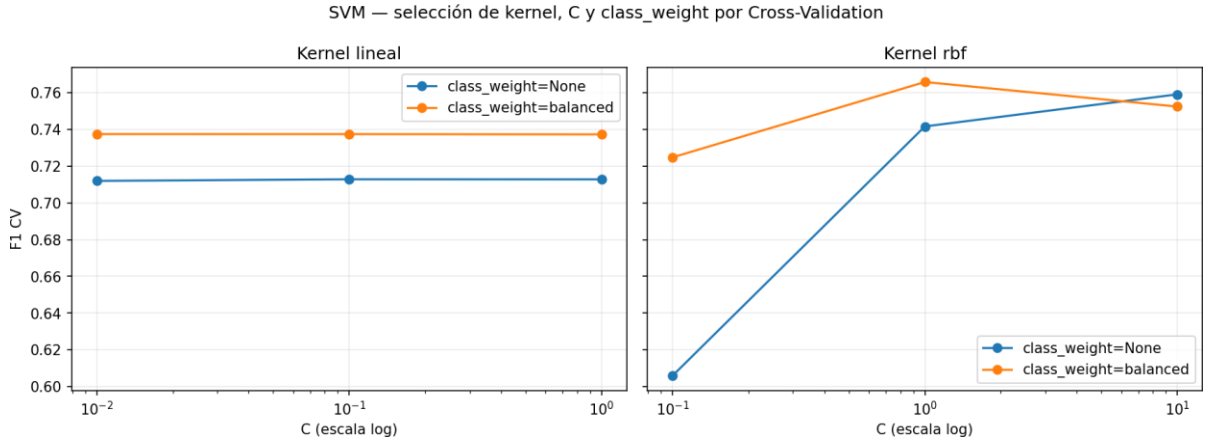


Figura 18. Selección de kernel, C y class_weight de la SVM mediante validación cruzada.

La Figura 18 muestra diferencias en el comportamiento de los dos enfoques evaluados. En el caso de LinearSVC, el F1-score se mantuvo prácticamente constante frente a los tres valores de C, lo que indica una baja sensibilidad del desempeño a este parámetro dentro del intervalo analizado. Asimismo, las configuraciones con class_weight='balanced' presentaron valores de F1 superiores a los obtenidos sin balanceo.

Por el contrario, el modelo con kernel RBF mostró una mayor variación del F1-score según el valor de C y el tratamiento del desbalance. La mejor configuración se obtuvo con C=1 y pesos balanceados, mientras que el incremento de C hasta 10 no produjo una mejora adicional bajo este tratamiento. Estos resultados evidencian que una mayor penalización de los errores de clasificación no garantiza un mejor desempeño de generalización y que el efecto de los pesos por clase depende de la configuración utilizada.

Una vez seleccionada la configuración RBF, se entrenó el modelo final utilizando una muestra estratificada y reproducible de 30.000 registros de TRAIN. Esta decisión respondió al costo computacional asociado con las SVM basadas en kernels, cuya complejidad de entrenamiento puede aumentar considerablemente con el tamaño del conjunto de datos. Aunque esta estrategia permitió reducir los requerimientos computacionales, también implicó que el modelo final no utilizara los 83.573 registros disponibles para entrenamiento.

Durante el ajuste final se identificaron 13.710 vectores de soporte, equivalentes aproximadamente al 45,7 % de los registros utilizados. Esta proporción indica que una cantidad

considerable de observaciones participó en la representación de la función de decisión. El resultado es compatible con un problema de separación complejo, en el que numerosas observaciones contribuyen a definir el clasificador. Sin embargo, la cantidad de vectores de soporte no permite demostrar por sí sola el grado de solapamiento entre las distribuciones de las clases, debido a que también depende del kernel, la regularización y las características de los datos.

Para ajustar el criterio de clasificación se utilizaron puntuaciones fuera de muestra obtenidas mediante validación cruzada sobre la muestra de 8.000 registros empleada en la selección de hiperparámetros. El procedimiento permitió establecer un umbral de aproximadamente +0,04137 sobre la función de decisión, utilizando la maximización del F1-score como criterio. Este valor no representa una probabilidad de cancelación, sino un punto de corte aplicado a las puntuaciones generadas por el clasificador.

Finalmente, el modelo entrenado con 30.000 registros se evaluó sobre los 35.817 registros del conjunto TEST. La Tabla 17 presenta las métricas obtenidas con el umbral previamente seleccionado.

Métrica	**Resultado**
Umbral	+0.041
Accuracy	0.8310
Balanced Accuracy	0.8208
Precision	0.7666
Recall	0.7817
F1-score	0.7741
ROC-AUC	0.9056
PR-AUC	0.8715

Tabla 17. Resultados TEST de la SVM con kernel RBF.

Los resultados de la Tabla 17 muestran que la Máquina de Soporte Vectorial con kernel RBF obtuvo un desempeño favorable en la clasificación de reservas hoteleras. Con el umbral de decisión de aproximadamente +0,041, el modelo alcanzó una Accuracy de 0,8310 y una Balanced Accuracy de 0,8208, lo que indica una capacidad adecuada para identificar ambas clases, considerando sus diferentes proporciones dentro del conjunto de datos.

En relación con la detección de cancelaciones, la SVM obtuvo una Precision de 0,7666 y

un Recall de 0,7817. Esto significa que aproximadamente el 76,66 % de las reservas clasificadas como canceladas efectivamente pertenecía a esta categoría, mientras que el modelo consiguió identificar el 78,17 % de las cancelaciones reales. La combinación de estas métricas produjo un F1-score de 0,7741, reflejando un equilibrio favorable entre la detección de cancelaciones y la precisión de las predicciones positivas.

Adicionalmente, se obtuvo un ROC-AUC de 0,9056 y un valor de Average Precision de 0,8715, presentado como PR-AUC en las tablas del proyecto. Estos resultados indican que las puntuaciones generadas por el modelo contienen información relevante para discriminar entre reservas canceladas y no canceladas a lo largo de diferentes puntos de corte.

Al comparar la SVM con los algoritmos desarrollados en la primera entrega, se observa que superó ligeramente al Árbol de Decisión podado, cuyo F1-score fue de 0,7708, y a KNN, que obtuvo 0,7592. Sin embargo, su desempeño fue inferior al de Random Forest ajustado, que alcanzó un F1-score de 0,7955 y un ROC-AUC de 0,9288. La diferencia de F1 entre la SVM y Random Forest fue de aproximadamente 0,0214, lo que sitúa a la SVM como una alternativa competitiva, aunque no como el clasificador de mayor desempeño dentro de las configuraciones evaluadas.

Desde la perspectiva computacional, la SVM con kernel RBF presentó un costo elevado en comparación con algunas alternativas. En la ejecución registrada, el entrenamiento final requirió aproximadamente 97,53 segundos y el cálculo de puntuaciones sobre TEST tomó alrededor de 58,16 segundos. Estos tiempos reflejan las condiciones específicas del equipo y del experimento, por lo que pueden variar en otras ejecuciones. La cantidad de vectores de soporte contribuye al costo de predicción, debido a que el modelo necesita evaluar la función de kernel respecto a numerosas observaciones almacenadas.

Una limitación adicional corresponde a que el umbral se seleccionó mediante predicciones fuera de muestra obtenidas a partir de 8.000 registros, mientras que el modelo final se entrenó con 30.000. Aunque ambos procedimientos utilizaron datos procedentes de TRAIN, las distribuciones de las puntuaciones pueden variar con el tamaño del conjunto de entrenamiento. En consecuencia, el umbral seleccionado no necesariamente representa el punto óptimo para el modelo ajustado sobre la muestra mayor, aspecto que debe considerarse al interpretar el F1 obtenido en TEST.

En conjunto, los resultados muestran que la SVM con kernel RBF constituye una alternativa capaz de representar relaciones de clasificación complejas y obtener un desempeño predictivo favorable. No obstante, su costo computacional, la cantidad de vectores de soporte y las limitaciones asociadas con el tamaño de entrenamiento y la selección del umbral deben considerarse al compararla con los demás algoritmos del proyecto.

Resultados de las tres redes neuronales

Para analizar el efecto de la complejidad arquitectónica sobre el desempeño en la clasificación de reservas hoteleras, se evaluaron tres redes neuronales artificiales mediante MLPClassifier: una red suave con una capa oculta de 16 neuronas, una red intermedia con dos capas ocultas de 64 neuronas cada una y una red grande con tres capas ocultas de 128 neuronas cada una. Estas configuraciones permitieron estudiar el comportamiento de arquitecturas progresivamente más complejas, manteniendo las mismas condiciones generales de preprocesamiento, entrenamiento y evaluación.

Inicialmente, las tres arquitecturas se evaluaron mediante validación cruzada estratificada de cinco particiones sobre una muestra reproducible de 20.000 registros de TRAIN. Durante este procedimiento se calcularon F1-score y ROC-AUC, considerando tanto sus valores medios como la variabilidad entre los folds. Posteriormente, se seleccionó para cada arquitectura un umbral de clasificación mediante predicciones fuera de muestra (*out-of-fold*) y se realizó el entrenamiento final utilizando los 83.573 registros del conjunto TRAIN, seguido de su evaluación sobre TEST. La Tabla 18 presenta las principales características y métricas obtenidas.

En validación cruzada, las tres arquitecturas mostraron resultados muy similares. La red suave presentó el mayor F1-score medio, con 0,7517 ± 0,0111, seguida por la red grande, con 0,7510 ± 0,0138, y la intermedia, con 0,7507 ± 0,0044. Las diferencias entre estos valores fueron inferiores a 0,001, considerablemente menores que la variabilidad registrada entre los folds. Por tanto, dentro de las condiciones evaluadas, no se identificaron mejoras relevantes en el F1-score medio al incrementar simultáneamente la cantidad de neuronas y capas ocultas.

En la evaluación final sobre TEST se observaron diferencias ligeramente mayores. La

red intermedia alcanzó el mayor F1-score, con 0,7870, seguida por la red grande, con 0,7815, y la red suave, con 0,7779. Sin embargo, estos resultados deben interpretarse considerando que los umbrales de decisión fueron seleccionados individualmente mediante validación cruzada. Además, el F1-score reportado durante la comparación inicial de arquitecturas se calculó utilizando el umbral predeterminado de 0,5, mientras que en TEST se utilizaron los umbrales optimizados de 0,38 para las redes suave e intermedia y de 0,34 para la red grande. En consecuencia, las diferencias entre F1 CV y F1 TEST no pueden atribuirse exclusivamente a cambios en la capacidad predictiva de los modelos.

Red	**Conexiones**	**F1 CV (± desv.)**	**Umbral**	**F1 TEST**	**Balanced Acc. TEST**	**ROC-AUC TEST**	**Respecto a la red anterior (CV)**
Suave (16)	1.600	0.7517 ± 0.0111	0.38	0.7779	0.8250	0.9173	Punto de partida
Intermedia (64, 64)	10.496	0.7507 ± 0.0044	0.38	0.7870	0.8314	0.9212	Quedó igual (−0.0010)
Grande (128, 128, 128)	45.568	0.7510 ± 0.0138	0.34	0.7815	0.8272	0.9192	Quedó igual (+0.0003)

Tabla 18. Comparación de las tres redes neuronales de clasificación.

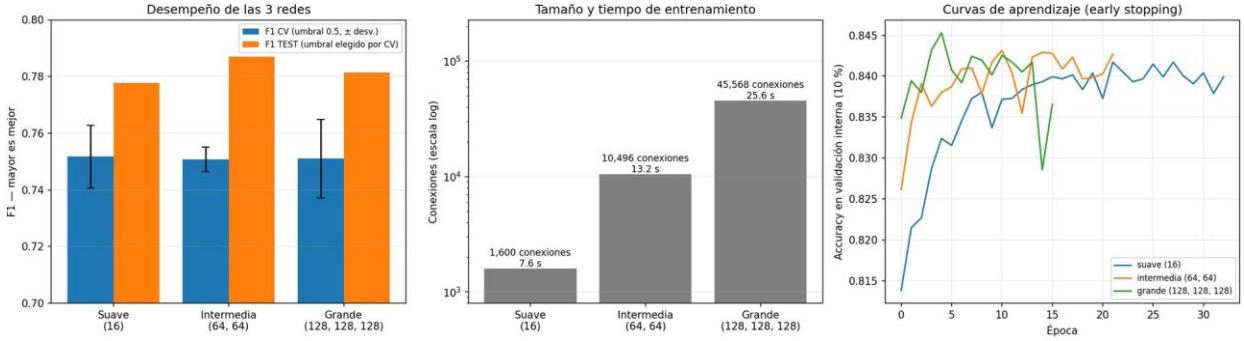


Figura 19. Desempeño, tamaño, tiempo de entrenamiento y curvas de aprendizaje de las tres redes de clasificación.

La Figura 19 permite analizar conjuntamente el desempeño predictivo, la complejidad arquitectónica y el costo computacional de las tres redes neuronales. En términos de conexiones, la red suave presentó 1.600, la intermedia 10.496 y la grande 45.568. Esto representa un incremento de aproximadamente 28,5 veces en el número de conexiones entre la arquitectura más sencilla y la más compleja. Sin embargo, este aumento no estuvo acompañado de una mejora consistente del F1-score de validación cruzada.

El costo computacional también aumentó con la complejidad arquitectónica. Los tiempos de entrenamiento registrados fueron de aproximadamente 6,87 segundos para la red suave, 12,51 segundos para la intermedia y 25,22 segundos para la grande. En consecuencia, la red grande requirió cerca de 3,7 veces el tiempo de entrenamiento de la red suave. Estas diferencias muestran que aumentar el tamaño de la arquitectura implica mayores requerimientos computacionales, aun cuando el desempeño predictivo se mantiene en valores similares. Los tiempos corresponden a las condiciones específicas de ejecución del experimento y pueden variar según los recursos disponibles.

Las curvas de aprendizaje presentadas en la tercera gráfica de la Figura 19 representan la evolución de Accuracy sobre la partición de validación interna utilizada por el mecanismo de detención temprana (*early stopping*). La red suave completó 33 épocas de entrenamiento, la intermedia 22 y la grande 16. Estas diferencias indican que las arquitecturas alcanzaron el criterio de detención en momentos distintos. No obstante, una menor cantidad de épocas no significa necesariamente un entrenamiento más eficiente, debido a que las arquitecturas más grandes requieren un mayor número de operaciones por época. Asimismo, estas curvas corresponden a la validación interna del entrenamiento final y no deben confundirse con el F1-score obtenido mediante validación cruzada.

Al comparar el desempeño de las tres arquitecturas, se observa que la red intermedia obtuvo el mayor F1-score sobre TEST, superando a la red suave en aproximadamente 0,0092 y a la grande en 0,0056. Sin embargo, esta ventaja no se reprodujo en la comparación inicial mediante validación cruzada, donde la red suave presentó un F1 medio ligeramente superior. Por tanto, el mejor resultado puntual de la red intermedia sobre TEST no constituye evidencia suficiente para establecer su superioridad general ni debe utilizarse como único criterio para seleccionarla.

Desde una perspectiva de selección de modelos, los resultados de validación cruzada y el costo computacional ofrecen argumentos a favor de la red suave entre las tres arquitecturas principales, debido a que obtuvo el mayor F1 medio de validación y presentó el menor número de conexiones y el menor tiempo de entrenamiento. Por otra parte, la red intermedia destacó por su menor desviación estándar entre folds y por alcanzar el mejor resultado descriptivo sobre TEST. Estas diferencias muestran que la elección depende del criterio priorizado, sin que los

resultados disponibles permitan afirmar una superioridad estadística concluyente de una arquitectura sobre las demás.

Para profundizar en el análisis, se evaluaron nueve combinaciones de arquitectura, incluidas las tres configuraciones principales, resultantes de combinar una, dos y tres capas ocultas con 16, 64 y 128 neuronas por capa. Este procedimiento permitió estudiar separadamente el efecto de incrementar el número de capas y el de aumentar las neuronas, manteniendo constante el otro factor. La Tabla 19 presenta los resultados obtenidos mediante validación cruzada.

Arquitectura	**Conexiones**	**F1 CV**	**Al agregar una capa**	**Al aumentar neuronas**
(16)	1.600	0.7517 ± 0.0111	—	—
(64)	6.400	0.7538 ± 0.0126	—	Quedó igual (+0.0021)
(128)	12.800	0.7574 ± 0.0082	—	Quedó igual (+0.0036)
(16, 16)	1.856	0.7504 ± 0.0106	Quedó igual (−0.0014)	—
(64, 64)	10.496	0.7507 ± 0.0044	Quedó igual (−0.0030)	Quedó igual (+0.0004)
(128, 128)	29.184	0.7480 ± 0.0131	Quedó igual (−0.0094)	Quedó igual (−0.0027)
(16, 16, 16)	2.112	0.7546 ± 0.0065	Quedó igual (+0.0042)	—
(64, 64, 64)	14.592	0.7451 ± 0.0073	Quedó igual (−0.0056)	Quedó igual (−0.0094)
(128, 128, 128)	45.568	0.7510 ± 0.0138	Quedó igual (+0.0030)	Quedó igual (+0.0059)

Tabla 19. Efecto de agregar capas y neuronas en las nueve combinaciones (clasificación).

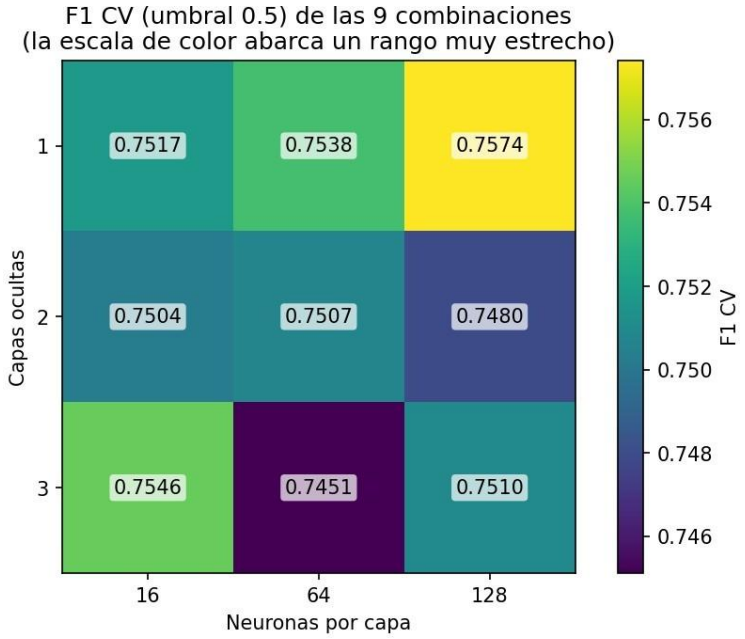


Figura 20. F1 CV de las nueve combinaciones de capas y neuronas.

La Figura 20 muestra que los valores de F1-score medio de las nueve combinaciones se encuentran aproximadamente entre 0,7451 y 0,7574, lo que evidencia diferencias relativamente reducidas entre las arquitecturas evaluadas. La configuración con una sola capa oculta y 128 neuronas presentó el mejor resultado, con un F1 medio de 0,7574 ± 0,0082, seguida por la arquitectura de tres capas con 16 neuronas por capa, que alcanzó 0,7546 ± 0,0065, y la configuración de una capa con 64 neuronas, cuyo F1 fue de 0,7538 ± 0,0126.

El análisis de las arquitecturas con una sola capa oculta muestra que incrementar las neuronas de 16 a 64 y posteriormente a 128 produjo pequeñas mejoras del F1 medio de validación cruzada. Sin embargo, dichas diferencias fueron inferiores a la variabilidad registrada entre folds. Por tanto, aunque se observó una tendencia favorable dentro de este grupo de configuraciones, los resultados no permiten establecer que el aumento del número de neuronas produzca una mejora consistente y generalizable.

Al analizar el incremento de capas ocultas manteniendo constante el número de neuronas por capa, tampoco se identificó un patrón uniforme de mejora. Algunas configuraciones mostraron incrementos pequeños del F1-score, mientras que otras presentaron disminuciones. Por ejemplo, con 64 neuronas por capa, el F1 medio pasó de 0,7538 con una capa a 0,7507 con dos capas y a 0,7451 con tres. Este comportamiento demuestra que una mayor profundidad no

necesariamente se traduce en una mejor capacidad predictiva dentro de las condiciones experimentales analizadas.

Es importante considerar que los resultados de la Figura 20 corresponden al F1-score obtenido mediante validación cruzada con el umbral predeterminado de 0,5. Asimismo, la escala de colores del mapa de calor abarca un intervalo reducido, por lo que las diferencias visuales entre las celdas pueden parecer más pronunciadas de lo que indican sus valores numéricos.

En conjunto, el análisis complementario muestra que incrementar la profundidad y el número de neuronas no produjo mejoras sistemáticas del desempeño predictivo. Aunque la arquitectura de una capa con 128 neuronas alcanzó el mayor F1 medio entre las nueve combinaciones, su ventaja fue pequeña frente a la variabilidad observada durante la validación. Estos resultados respaldan la importancia de evaluar diferentes arquitecturas mediante validación cruzada y considerar conjuntamente el desempeño predictivo y la complejidad computacional, evitando asumir que una red neuronal de mayor tamaño será necesariamente superior.

Comparación final de modelos

Con el propósito de establecer una comparación integral del desempeño predictivo de los algoritmos implementados en ambas entregas, se evaluaron los clasificadores finales sobre el mismo conjunto TEST, compuesto por 35.817 reservas hoteleras. La comparación incluyó los tres algoritmos supervisados de la primera entrega, los seis modelos o configuraciones principales incorporados en la segunda y el clasificador de referencia basado en la clase mayoritaria.

Para mantener criterios comunes de evaluación, se utilizaron las métricas Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y Average Precision. Se estableció el F1-score como criterio principal de comparación, debido a que permite considerar conjuntamente la capacidad para identificar cancelaciones reales y la precisión de las predicciones positivas. Asimismo, cada clasificador utilizó el umbral de decisión seleccionado previamente mediante validación cruzada dentro de TRAIN, sin realizar ajustes adicionales a partir de los resultados de TEST.

Aunque los modelos se evaluaron sobre las mismas observaciones, sus procedimientos de entrenamiento presentaron diferencias relacionadas con los algoritmos, el preprocesamiento y los tamaños de muestra utilizados. Por tanto, los resultados permiten comparar el desempeño de las configuraciones efectivamente implementadas, pero no aislar completamente el efecto de cada familia de modelos. La Tabla 20 presenta los resultados finales, ordenados de mayor a menor F1-score.

Modelo	**Entrega**	**Accuracy**	**Balanced Acc.**	**Precision**	**Recall**	**F1**	**ROC-AUC**	**PR-AUC**
Random Forest ajustado	E1	0.8463	0.8382	0.7842	0.8071	0.7955	0.9288	0.8986
Red neuronal intermedia	E2	0.8397	0.8314	0.7748	0.7997	0.7870	0.9212	0.8901
Red neuronal grande	E2	0.8341	0.8272	0.7632	0.8006	0.7815	0.9192	0.8879
Red neuronal suave	E2	0.8277	0.8250	0.7445	0.8144	0.7779	0.9173	0.8853
SVM RBF (C = 1)	E2	0.8310	0.8208	0.7666	0.7817	0.7741	0.9056	0.8715
Árbol podado	E1	0.8362	0.8172	0.7999	0.7438	0.7708	0.9128	0.8692
KNN ajustado	E1	0.8235	0.8086	0.7674	0.7512	0.7592	0.9014	0.8696
Red Bayesiana (11 atributos)	E2	0.8182	0.8039	0.7578	0.7485	0.7531	0.8993	0.8617
Naive Bayes categórico	E2	0.7579	0.7505	0.6579	0.7219	0.6884	0.8308	0.7957
Baseline, clase mayoritaria	—	0.6296	0.5000	0.0000	0.0000	0.0000	0.5000	0.3704

Tabla 20. Comparación final de los modelos de ambas entregas sobre el mismo conjunto TEST (clasificación).

Los resultados de la Tabla 20 muestran que Random Forest ajustado, desarrollado en la primera entrega, conservó el mejor desempeño global de clasificación según el criterio principal establecido. Este modelo obtuvo un F1-score de 0,7955, una Balanced Accuracy de 0,8382, un ROC-AUC de 0,9288 y una Average Precision de 0,8986. En conjunto, estas métricas evidencian un desempeño favorable tanto en la clasificación de reservas mediante el umbral seleccionado como en la capacidad de discriminar entre cancelaciones y no cancelaciones.

Entre los algoritmos incorporados en la segunda entrega, las redes neuronales obtuvieron los mejores resultados. La arquitectura intermedia alcanzó un F1-score de 0,7870, seguida por la grande, con 0,7815, y la suave, con 0,7779. La diferencia absoluta entre Random Forest y la red

intermedia fue de aproximadamente 0,0085, equivalente a 0,85 puntos porcentuales de F1. Esta diferencia es pequeña en magnitud, por lo que, aunque Random Forest obtuvo el mejor resultado puntual, no es posible afirmar una superioridad estadística concluyente sin realizar comparaciones adicionales sobre las predicciones de los modelos.

La SVM con kernel RBF alcanzó un F1-score de 0,7741, superando ligeramente al Árbol de Decisión podado, cuyo resultado fue de 0,7708, y a KNN, que obtuvo 0,7592. Sin embargo, el Árbol podado presentó un ROC-AUC de 0,9128, superior al 0,9056 obtenido por la SVM. Esta diferencia demuestra que el orden de los modelos puede cambiar según la métrica utilizada, debido a que F1 depende del umbral de clasificación seleccionado, mientras que ROC-AUC evalúa la capacidad discriminativa a lo largo de diferentes puntos de corte.

En cuanto a los modelos bayesianos de clasificación, la Red Bayesiana obtuvo un F1-score de 0,7531 y un ROC-AUC de 0,8993, superando al Naive Bayes categórico, que alcanzó 0,6884 y 0,8308, respectivamente. Estos resultados muestran un desempeño favorable de la Red Bayesiana respecto a la configuración de Naive Bayes evaluada, aunque ambos modelos quedaron por debajo de los algoritmos con mejores resultados de clasificación. No obstante, la Red Bayesiana presentó una ventaja relacionada con la representación explícita de dependencias probabilísticas y la posibilidad de realizar consultas condicionadas sobre las características de las reservas.

Todos los clasificadores supervisados superaron al modelo de referencia basado en la clase mayoritaria, cuya Accuracy fue de 0,6296, Balanced Accuracy de 0,5000 y F1-score de 0 para la clase cancelada. Este resultado evidencia que los algoritmos desarrollados consiguieron identificar patrones predictivos que permiten reconocer cancelaciones, a diferencia de una estrategia que asigna todas las reservas a la categoría predominante.

Desde la perspectiva de selección de modelos, los resultados muestran que Random Forest conserva el mejor desempeño predictivo puntual, mientras que las redes neuronales ofrecen resultados cercanos y pueden representar alternativas computacionalmente favorables en las condiciones evaluadas. Sin embargo, no debe seleccionarse una arquitectura exclusivamente por su posición en TEST, sino mediante los criterios previamente definidos durante el entrenamiento y la validación cruzada. Por tanto, la recomendación final debe considerar conjuntamente las métricas predictivas, la estabilidad experimental, la

interpretabilidad y los requerimientos computacionales.

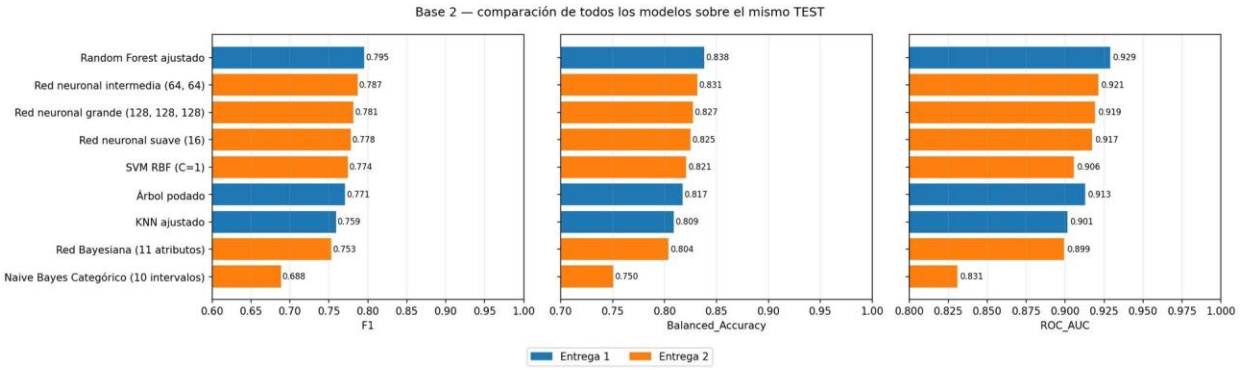


Figura 21. Comparación final del F1, la Balanced Accuracy y el ROC-AUC de todos los modelos sobre TEST.

La Figura 21 permite visualizar las diferencias de desempeño entre los algoritmos de ambas entregas considerando tres criterios complementarios. Random Forest ocupa la primera posición en F1-score, Balanced Accuracy y ROC-AUC, mientras que las redes neuronales presentan valores cercanos y un comportamiento competitivo en las tres métricas. Por su parte, la SVM con kernel RBF obtiene un F1 ligeramente superior al Árbol podado, aunque este último alcanza un mayor ROC-AUC, evidenciando que el desempeño relativo de los clasificadores depende del criterio de evaluación utilizado.

Las diferencias entre los modelos mejor posicionados son relativamente reducidas, especialmente entre Random Forest y las redes neuronales. Además, los ejes de las gráficas utilizan escalas ampliadas para facilitar la visualización de estas diferencias, por lo que la longitud de las barras debe interpretarse considerando los valores numéricos reportados. En conjunto, la figura confirma la importancia de utilizar varias métricas complementarias, en lugar de fundamentar la comparación exclusivamente en una medida de desempeño.

Las curvas ROC y Precision-Recall de la figura 22 confirman el orden de forma independiente del umbral: las curvas de Random Forest y de las redes neuronales quedan por encima del resto, y la de Naive Bayes es la más cercana a la línea de azar.

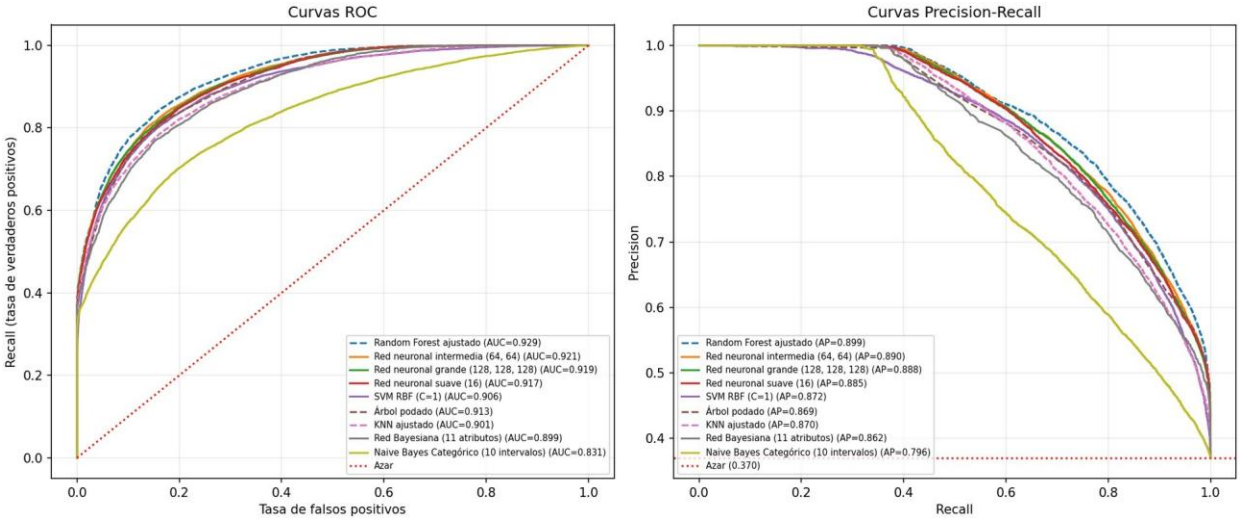


Figura 22. Curvas ROC y Precision-Recall de todos los modelos sobre TEST.

En la curva ROC se observa que Random Forest presenta el mayor valor del área bajo la curva, con 0,9288, seguido por las redes neuronales, cuyos resultados se encuentran entre 0,9173 y 0,9212. Esto indica que estos algoritmos presentan la mayor capacidad discriminativa global entre las configuraciones evaluadas. Por su parte, Naive Bayes presenta el menor ROC-AUC, con 0,8308, aunque mantiene un desempeño considerablemente superior al de un clasificador aleatorio, representado por la diagonal de referencia.

La curva Precision-Recall muestra un comportamiento semejante en términos generales. Random Forest obtuvo la mayor Average Precision, con 0,8986, seguido por las redes neuronales, con valores comprendidos entre 0,8853 y 0,8901. La SVM RBF alcanzó 0,8715 y la Red Bayesiana 0,8617, mientras que Naive Bayes presentó el menor resultado entre los clasificadores supervisados, con 0,7957. No obstante, este último valor continúa siendo superior a la prevalencia de la clase positiva, utilizada como referencia de un clasificador sin capacidad de ordenamiento.

Aunque las curvas permiten identificar tendencias generales, no debe asumirse que los modelos mantienen exactamente el mismo orden en todos los niveles de Recall o de tasa de falsos positivos. Algunas curvas se aproximan y pueden presentar intersecciones, mientras que sus métricas globales resumen el desempeño a lo largo de diferentes puntos de corte. Asimismo, la Average Precision utilizada en este proyecto corresponde al cálculo implementado por average_precision_score de Scikit-learn, por lo que no debe confundirse con una integración trapezoidal de la curva Precision-Recall.

En conjunto, ambas representaciones muestran que Random Forest y las redes neuronales

presentan una mayor capacidad general de discriminación, mientras que los modelos bayesianos, especialmente Naive Bayes, obtienen resultados inferiores. Sin embargo, la elección del clasificador debe considerar también el comportamiento de los errores concretos de predicción, debido a que las métricas globales no muestran directamente cuántas cancelaciones se dejan de identificar ni cuántas reservas no canceladas son clasificadas incorrectamente. Para complementar este análisis, se examinan las matrices de confusión de los modelos incorporados en la segunda entrega.

Finalmente, se analizaron las matrices de confusión de los seis clasificadores incorporados en la segunda entrega, utilizando los umbrales de decisión seleccionados mediante validación cruzada. Estas matrices permiten examinar la distribución de los aciertos y errores de cada modelo, diferenciando los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos. En este problema, los verdaderos positivos representan reservas canceladas correctamente identificadas, mientras que los falsos negativos corresponden a cancelaciones reales que el modelo no consiguió detectar. Por su parte, los falsos positivos representan reservas que no fueron canceladas, pero que el clasificador identificó incorrectamente como canceladas.

La Figura 23 presenta las matrices correspondientes a las tres redes neuronales, la SVM con kernel RBF, la Red Bayesiana y Naive Bayes categórico. Todas fueron calculadas utilizando los mismos 35.817 registros de TEST, compuestos por 22.550 reservas no canceladas y 13.267 canceladas, lo que permite comparar directamente la distribución de los errores de clasificación.

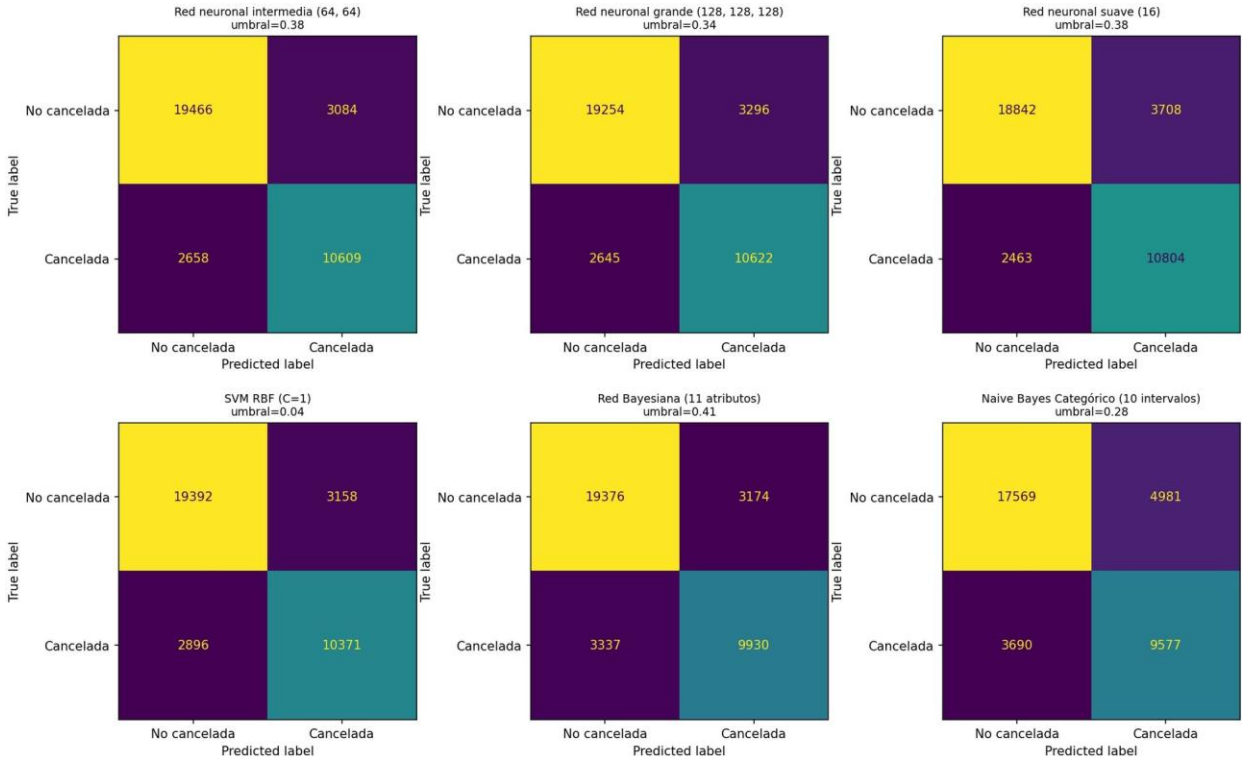


Figura 23. Matrices de confusión de los modelos de la segunda entrega sobre TEST.

El análisis de las matrices de confusión muestra diferencias relevantes entre los modelos en cuanto a la identificación de cancelaciones y la generación de falsos positivos. Entre las tres redes neuronales, la arquitectura suave identificó correctamente 10.804 cancelaciones y presentó 2.463 falsos negativos, obteniendo el mayor Recall de las configuraciones incorporadas en la segunda entrega (0,8144). Sin embargo, también generó 3.708 falsos positivos, una cantidad superior a la registrada por las redes intermedia y grande.

La red neuronal intermedia identificó correctamente 10.609 cancelaciones, con 2.658 falsos negativos y 3.084 falsos positivos. Aunque detectó menos cancelaciones que la red suave, redujo considerablemente las reservas no canceladas clasificadas incorrectamente como canceladas. Este comportamiento explica que alcanzara una Precision superior y el mayor F1-score entre los modelos nuevos, con 0,7870. Por su parte, la red grande obtuvo 10.622 verdaderos positivos, 2.645 falsos negativos y 3.296 falsos positivos, mostrando un comportamiento intermedio entre las otras dos arquitecturas.

La SVM con kernel RBF identificó correctamente 10.371 cancelaciones y registró 2.896 falsos negativos y 3.158 falsos positivos. Estos resultados representan un equilibrio favorable entre

las dos clases, aunque su F1-score fue inferior al de las tres redes neuronales. Por su parte, la Red Bayesiana obtuvo 9.930 verdaderos positivos, 3.337 falsos negativos y 3.174 falsos positivos, evidenciando una capacidad de detección de cancelaciones menor que la de las redes neuronales y la SVM.

Naive Bayes presentó las mayores dificultades entre los clasificadores evaluados. El modelo identificó correctamente 9.577 cancelaciones, pero registró 3.690 falsos negativos y 4.981 falsos positivos. Esta última cifra fue considerablemente superior a la de los demás algoritmos, lo que explica parcialmente su menor Precision y F1-score. Aunque el modelo conserva capacidad predictiva, presenta mayores dificultades para distinguir correctamente las reservas canceladas de las no canceladas con el umbral seleccionado.

En conjunto, las matrices permiten observar que los clasificadores presentan diferentes equilibrios entre los dos tipos de error. La red suave favoreció una mayor detección de cancelaciones, aunque con un incremento de falsos positivos, mientras que la red intermedia consiguió una relación más equilibrada entre ambos indicadores. Estas diferencias son relevantes porque, en una aplicación hotelera, clasificar incorrectamente una reserva como cancelada y no detectar una cancelación real pueden generar consecuencias operativas distintas. Sin embargo, el proyecto no establece costos económicos específicos para estos errores, por lo que no es posible determinar cuál de ellos debería recibir mayor penalización en un escenario de implementación real.

Por tanto, el análisis de las matrices de confusión complementa las métricas globales y demuestra que la selección de un clasificador no debe fundamentarse únicamente en Accuracy o F1-score, sino también en el tipo de errores que produce y en las necesidades del problema que se pretende resolver.

CONCLUSIÓN

El desarrollo de la segunda entrega permitió ampliar el análisis de los problemas de regresión y clasificación abordados inicialmente, mediante la implementación de modelos bayesianos, Máquinas de Soporte Vectorial y Redes Neuronales Artificiales. La conservación de los conjuntos de datos, las particiones Hold-Out y los criterios generales de evaluación permitió establecer una comparación entre los algoritmos desarrollados en ambas entregas. Asimismo, la reproducción de las métricas de los modelos anteriores proporcionó evidencia de consistencia experimental, aunque las diferencias en los procedimientos y tamaños de entrenamiento de ciertos algoritmos deben considerarse al interpretar los resultados.

En el problema de regresión, orientado a estimar el tiempo de reacción de jugadores de e-sports, los modelos incorporados en la segunda entrega obtuvieron menores errores de predicción que los algoritmos evaluados previamente. La red neuronal grande alcanzó el menor RMSE sobre TEST, con 14,9956 ms y un R^2 de 0,3784, frente a los 15,1495 ms obtenidos por Random Forest, el mejor modelo de la primera entrega. No obstante, la red neuronal suave consiguió un resultado prácticamente idéntico, con un RMSE de 14,9960 ms, utilizando una arquitectura considerablemente menos compleja. Por su parte, la regresión lineal bayesiana y la SVR lineal alcanzaron errores cercanos a 15,06 ms, demostrando que los modelos de menor complejidad también pueden ofrecer un desempeño competitivo.

La comparación mediante validación cruzada permitió complementar estos resultados, identificando a la regresión bayesiana como el modelo con menor RMSE medio dentro de la evaluación común. Asimismo, el mejor comportamiento de las configuraciones lineales de SVR frente a las RBF sugiere que las relaciones lineales representan una parte importante de los patrones predictivos presentes en el conjunto de datos. Sin embargo, estos hallazgos no demuestran que el fenómeno sea exclusivamente lineal ni permiten descartar mejoras mediante otras configuraciones o variables. Además, los valores de R^2, cercanos a 0,38 en los mejores modelos, muestran que una proporción considerable de la variabilidad del tiempo de reacción permanece sin explicar.

La regresión bayesiana aportó una ventaja adicional relacionada con la interpretación de los coeficientes y la cuantificación de la incertidumbre predictiva. Las horas de sueño, el tiempo

dedicado al juego y la edad presentaron las mayores asociaciones dentro de las variables numéricas estandarizadas. Asimismo, los intervalos predictivos aproximados del 95 % alcanzaron una cobertura empírica del 95,1 % sobre TEST, resultado compatible con una adecuada cobertura global bajo las condiciones evaluadas. No obstante, estas asociaciones no deben interpretarse como relaciones causales demostradas, ni la cobertura observada garantiza que el modelo conserve el mismo comportamiento en otros conjuntos de datos.

En el problema de clasificación de reservas hoteleras, Random Forest ajustado, desarrollado en la primera entrega, conservó el mejor desempeño global según el F1-score, con un valor de 0,7955 y un ROC-AUC de 0,9288. Entre los nuevos algoritmos, la red neuronal intermedia obtuvo el mayor F1 sobre TEST, con 0,7870, seguida por las redes grande y suave, con 0,7815 y 0,7779, respectivamente. La SVM con kernel RBF también presentó un desempeño competitivo, alcanzando un F1 de 0,7741. Estos resultados evidencian que las redes neuronales y las Máquinas de Soporte Vectorial pueden representar adecuadamente patrones de clasificación complejos, aunque ninguna de las configuraciones nuevas superó el resultado final de Random Forest.

El análisis de los modelos bayesianos permitió examinar la importancia de las relaciones probabilísticas entre las características de las reservas. La evaluación mediante la V de Cramér identificó asociaciones entre determinados atributos dentro de las clases de cancelación, lo que evidencia limitaciones del supuesto de independencia condicional utilizado por Naive Bayes. En la comparación realizada sobre los mismos once atributos discretizados, la Red Bayesiana obtuvo un F1 de 0,7531, superior al 0,7011 alcanzado por Naive Bayes. Asimismo, presentó una mayor capacidad discriminativa y permitió representar dependencias mediante un grafo dirigido y realizar consultas probabilísticas sobre diferentes condiciones de reserva. Sin embargo, la mejora no puede atribuirse exclusivamente a la estructura aprendida, debido a las diferencias entre los procedimientos de estimación de ambos modelos. Las conexiones de la Red Bayesiana deben interpretarse como relaciones probabilísticas y no como vínculos causales demostrados.

Respecto a las Redes Neuronales Artificiales, la comparación de las tres arquitecturas principales y las nueve combinaciones complementarias mostró que incrementar el número de neuronas y capas ocultas no produjo mejoras consistentes del desempeño predictivo. En regresión, las tres redes obtuvieron errores prácticamente iguales, mientras que la cantidad de

conexiones y los tiempos de entrenamiento aumentaron considerablemente. En clasificación, la red intermedia alcanzó el mayor F1 sobre TEST entre las tres arquitecturas principales, pero la red suave obtuvo un F1 medio ligeramente superior durante la validación cruzada. Además, en el análisis complementario, la arquitectura de una sola capa con 128 neuronas presentó el mayor F1 medio entre las nueve combinaciones. Estas diferencias evidencian que una mayor complejidad arquitectónica no garantiza una mejor capacidad de generalización y que la selección de una red debe considerar tanto las métricas obtenidas durante la validación como sus requerimientos computacionales.

Desde el punto de vista metodológico, los experimentos también permitieron identificar limitaciones que deben considerarse al interpretar las comparaciones. Algunos algoritmos requirieron utilizar muestras reducidas durante la búsqueda de hiperparámetros o el entrenamiento final, debido a su costo computacional. Asimismo, en determinados casos se utilizaron estimadores diferentes entre la selección inicial y el ajuste final, como ocurrió con SVR y LinearSVR. En la Red Bayesiana, el aislamiento del preprocesamiento y de los catálogos de estados dentro de cada partición de validación cruzada requiere una verificación adicional. Estas condiciones no invalidan automáticamente los resultados reportados, pero limitan la posibilidad de atribuir todas las diferencias observadas exclusivamente a las características de los algoritmos. Adicionalmente, no se realizaron pruebas formales de significancia estadística que permitan establecer la superioridad concluyente de los modelos cuyos desempeños fueron muy próximos.

En conjunto, los resultados de ambas entregas permiten concluir que la selección de un algoritmo de aprendizaje automático no debe depender únicamente de su complejidad o de una métrica obtenida sobre TEST. En regresión, la red neuronal suave y la regresión bayesiana constituyen alternativas favorables por su equilibrio entre desempeño predictivo, simplicidad e interpretabilidad, según el criterio priorizado. En clasificación, Random Forest conserva el mejor desempeño final, mientras que las redes neuronales ofrecen resultados competitivos y la Red Bayesiana aporta herramientas adicionales de interpretación probabilística. Por tanto, la elección de un modelo debe fundamentarse en una evaluación integral de su precisión, capacidad de generalización, estabilidad, interpretabilidad y costo computacional, considerando tanto las características del problema como las limitaciones de los datos y del procedimiento

experimental.